

RÉPUBLIQUE DU CAMEROUN
Paix - Travail - Patrie

UNIVERSITÉ DE YAOUNDÉ I

CENTRE DE RECHERCHE ET DE
FORMATION DOCTORALE EN SCIENCES,
TECHNOLOGIES ET GEOSCIENCES

Unité de recherche et de formation
doctorale en mathématiques,
informatique, bio-informatiques et
applications

BP 812 Yaoundé



REPUBLIC OF CAMEROON
Peace - Work - Fatherland

UNIVERSITY OF YAOUNDE I

POST GRADUATED SCHOOL OF
SCIENCES, TECHNOLOGY AND
GEOSCIENCES

Doctoral research unit for
mathematics, computer sciences
and applications

PO Box 812 Yaounde

DÉPARTEMENT D'INFORMATIQUE

DEPARTMENT OF COMPUTER SCIENCE

LABORATOIRE INFORMATIQUE ET APPLICATIONS

LABORATORY OF INFORMATICS AND APPLICATIONS

Mémoire soumis en vue de l'obtention du diplôme de Master en Informatique Fondamentale

Option : Systèmes et Réseaux

ALLOCATION DES RESSOURCES DANS LE CALCUL SUR MACHINES VOLONTAIRES VIA L'APPRENTISSAGE PAR RENFORCEMENT

Soutenu par :

NOAH MVONDO Serge Gaetan

Matricule : 20U2669

Sous la supervision de :

ADAMOU Hamza

Chargé de cours

Université de Yaoundé I

Année académique : 2024-2025

Dédicaces

Je dédie ce travail à mes parents

Remerciements

Je rends grâce au Tout-Puissant pour avoir été présent tout au long de l'accomplissement de ce travail. La réalisation de ce travail a nécessité la contribution de plusieurs personnes auxquelles je tiens à exprimer toute ma gratitude.

Je remercie en tout premier lieu les membres de mon jury de soutenance, pour l'honneur octroyé en acceptant d'évaluer ce travail.

Je remercie mon encadreur, le Dr Hamza ADAMOU, pour m'avoir donné l'opportunité de travailler sur cette thématique et pour m'avoir guidé tout au long de ce mémoire.

Je remercie l'unité de recherche UMMISCO pour cette période d'incubation en stage de recherche.

Je remercie le professeur Norbert TSOPZE pour les nombreux échanges, et les précieux conseils qu'il m'a prodigué.

Je remercie mes aînés GBETNKOM NJIFON Jeff et KITWE ADAGAO pour nos échanges, les encouragements et l'aide qu'ils m'ont apportés au quotidien jusqu'à l'aboutissement de ce travail.

Je remerci M. Lionel NAGEU NDJAMBONG de TOGETTECH pour les échanges en lignes, les conseils sur la recherche et le soutien.

Je remercie mes parents feu M. MVONDO Jean Marie et Mme MVONDO Généviève, sans qui, rien de tout ceci n'aurait été possible.

Je remercie M. et Mme ETOUNDI, M. et Mme NDONGO, M. et Mme NKÉ pour leur accueil, leur hospitalité et pour tout leur apport durant ces années. Je remercie tous mes proches, spécialement Juliette NDZIÉ MVONDO, Thérèse MOLO EDOUMA, Sylvianne NAMA et mon frère CHARLY pour l'accompagnement et le soutien indéfectible.

Je remercie mes amis, spécialement TOKO KEMZANG, DEMINANG VIANNEY, Robinson, Kaptchuang Derick, EMerson, Homer, Eric OKALA, pour nos échanges, les encouragements et l'aide qu'ils m'ont apportés au quotidien jusqu'à l'aboutissement de ce travail.

Je remercie les étudiants du département d'informatique pour leurs questions et leur participation tout au long de la réalisation de ce travail.

Je remercie tous ceux que je n'ai pas encore remercié et qui, de près ou de loin, ont contribué à la réalisation de ce travail.

Merci.

Résumé

Le calcul sur machines volontaires exploite les ressources inactives des ordinateurs personnels pour effectuer des calculs scientifiques à grande échelle, offrant une alternative économique aux superordinateurs. Cependant, des défis tels que l'hétérogénéité matérielle et logicielle, la volatilité des nœuds et les dépendances des tâches compliquent l'allocation des ressources. Les méthodes d'ordonnancement traditionnelles, allant des approches naïves comme FCFS aux techniques basées sur la connaissance comme MET et les modèles probabilistes, manquent souvent d'adaptabilité dans des environnements dynamiques, entraînant une augmentation du makespan, des taux d'échec et une utilisation inefficace des ressources. Ce mémoire propose une stratégie d'ordonnancement basée sur l'apprentissage par renforcement pour surmonter ces limitations. Nous modélisons le problème comme un Processus de Décision Markovien, où les états capturent les statuts des tâches, la disponibilité des volontaires et les métriques de ressources ; les actions impliquent l'assignation de tâches ou leur report ; et les récompenses équilibrent la réduction du makespan, l'amélioration de l'utilisation et la minimisation des coûts réseau. En utilisant l'algorithme Asynchronous Advantage Actor-Critic, nous avons entraîné notre agent dans un environnement simulé avec des profils de volontaires réalistes et des workflows. Puis, nous avons testé notre approche dans le système de calcul volontaire de l'Université de Yaoundé I (VCUY1), elle a démontré des performances supérieures en scénarios de forte charge : une réduction de 16,3 % du makespan par rapport à FCFS et 7,8 % versus MET, avec une utilisation plus élevée (89,23 %) et des taux d'échec plus bas (3,2 %). Ce travail contribue une formalisation MDP adaptative et une intégration A3C dans un système de calcul volontaire, avec des évaluations montrant des gains en contextes volatils. Les perspectives incluent des tests à plus grande échelle, des méthodes hybrides heuristique-RL, l'optimisation des coûts d'entraînement et l'intégration de GPU.

Mots-clés : Calcul Volontaire, Allocation de Ressources, Ordonnancement, Apprentissage par Renforcement, A3C, MDP.

Abstract

Volunteer computing harnesses the idle resources of personal computers to perform large-scale scientific computations, providing a cost-effective alternative to supercomputers. Despite its potential, several challenges complicate efficient resource allocation, including hardware and software heterogeneity, node volatility, and task dependencies. Traditional scheduling approaches—ranging from naïve strategies such as First-Come, First-Served (FCFS) to knowledge-based methods like Minimum Execution Time (MET) and probabilistic models—often lack the adaptability required in dynamic environments. This typically results in increased makespan, higher failure rates, and suboptimal resource utilization. In this thesis, we introduce a reinforcement learning-based scheduling strategy designed to address these limitations. The scheduling problem is formulated as a Markov Decision Process (MDP), where states represent task progress, volunteer availability, and resource metrics; actions consist of assigning or deferring tasks; and rewards balance multiple objectives, including reducing makespan, improving utilization, and minimizing communication overhead. Our agent, trained with the Asynchronous Advantage Actor-Critic (A3C) algorithm in a simulated environment using realistic volunteer profiles and workflows, was subsequently evaluated within the University of Yaoundé I Volunteer Computing System (VCUY1). Under high-load scenarios, our approach achieved notable improvements: a 16.3% reduction in makespan compared to FCFS and 7.8% relative to MET, alongside higher utilization (89.23%) and reduced failure rates (3.2%). This work contributes an adaptive MDP formalization and an A3C-based scheduling framework for volunteer computing, with experimental results highlighting its robustness in volatile environments. Future directions include large-scale evaluations, hybrid heuristic-RL approaches, optimization of training overhead, and GPU integration.

Keywords : Volunteer Computing, Resource Allocation, Scheduling, Reinforcement Learning, A3C, MDP.

Table des matières

Dédicaces	i
Remerciements	i
Résumé	ii
Abstract	iii
Liste des figures	vi
Liste des tableaux	vii
Liste des algorithmes	viii
Introduction générale	1
1 Etat de l’art	3
Introduction	3
1.1 Généralités sur le calcul sur machines volontaires	3
1.1.1 Définition et objectifs	4
1.1.2 Architectures	4
1.1.3 Stratégies pour l’amélioration des performances	5
1.2 Méthodes d’allocation des ressources dans le calcul sur machines volontaires	6
1.2.1 Définition et rôle de l’ordonnancement	7
1.2.2 Méthodes d’allocation des ressources	7
1.2.3 Tableau récapitulatif et comparatif	10
1.2.4 Limites des méthodes traditionnelles	10
1.3 Apprentissage par renforcement pour un ordonnancement adaptatif	11
1.3.1 Présentation du RL pour l’ordonnancement	11
1.3.2 Modélisation du problème d’ordonnancement comme un MDP . . .	12
1.3.3 Algorithmes de RL	12
1.3.4 Application de A3C dans le volunteer edge-cloud computing	13
1.3.5 Synthèse	14
Conclusion	15
2 Approche d’ordonnancement de tâches dans le calcul volontaire utilisant l’apprentissage par renforcement.	16
Introduction	16
2.1 Modélisation du Problème d’Ordonnancement	17



2.1.1	Définition des Entités et Contraintes du Système	17
2.1.2	Modélisation de la Volatilité et Disponibilité	18
2.1.3	Scores de Satisfaction pour l'Évaluation des Assignations	19
2.2	Solution basée sur Apprentissage par Renforcement	22
2.2.1	Formalisation sous un Processus de Décision Markovien (MDP)	23
2.2.2	Architecture A3C Adaptée	23
2.3	Solution Algorithmique et Perspectives	25
2.3.1	Algorithme d'Apprentissage	25
2.3.2	Synthèse et Avantages de l'Approche	27
	Conclusion	27
3	Expérimentation et résultats	28
	Introduction	28
3.1	Méthodologie expérimentale	28
3.1.1	Conception de l'expérimentation	29
3.1.2	Protocole expérimental	30
3.2	Mise en œuvre de l'approche proposée	31
3.2.1	Système utilisé	31
3.2.2	Workflows	31
3.2.3	Caractéristiques des volontaires	32
3.3	Résultats obtenus et interprétation	32
3.3.1	Présentation des résultats	32
3.3.2	Comparaison des résultats	33
3.3.3	Interprétation et discussion	34
	Conclusion	35
	Conclusion et Perspectives	37



Table des figures

3.1	Comparaison des métriques pour la charge faible.	33
3.2	Comparaison des métriques pour la charge moyenne.	34
3.3	Comparaison des métriques pour la charge forte.	34

Liste des tableaux

1.1	Comparaison des méthodes d'allocation de ressources dans le calcul volontaire	10
1.2	Synthèse des approches DRL pour l'ordonnancement des tâches dans le cloud	14
3.1	Répartition par type de machine et RAM	32
3.2	Resumé des amelioration de notre approche	35

Liste des algorithmes

1	A3C Adapté pour l'Ordonnancement avec Gestion de la Volatilité et Hétérogénéité	26
---	---	----

Introduction Générale

Le paysage scientifique contemporain est marqué par une explosion des données et des besoins en calcul intensif, propulsée par des domaines comme la biologie computationnelle, la physique des particules et l'intelligence artificielle. Cependant, l'accès à des superordinateurs reste limité par des coûts prohibitifs et une infrastructure centralisée. Parallèlement, des milliards d'ordinateurs personnels à travers le monde restent sous-utilisés, souvent à moins de 15 % de leur capacité. Le calcul sur machines volontaires (Volunteer Computing, VC) émerge comme une solution choisie, exploitant ces ressources inactives via des réseaux distribués pour former une puissance de calcul collective, économique et écologique. Ce paradigme, illustré par des projets comme SETI@home ou BOINC, permet d'assembler des milliers, voire des millions de machines personnelles offertes par des volontaires.

Étant donné que les systèmes de calcul sur machines volontaires reposent sur la contribution volontaire de ressources, les participants peuvent se déconnecter à tout moment sans préavis, engendrant une volatilité significative des nœuds. De plus, les machines des volontaires, caractérisées par une grande diversité matérielle (e.g., CPU, mémoire) et logicielle (e.g., systèmes d'exploitation), introduisent une hétérogénéité marquée. Cette variabilité soulève des défis critiques en termes de sécurité, de fiabilité et de stabilité des performances, entraînant des dépassements de délais, des résultats erronés et des taux d'échec élevés [1]. Pour répondre à ces contraintes, la littérature propose des mécanismes tels que la réplication des tâches, la migration entre nœuds et l'ordonnancement adaptatif, ce dernier étant central pour optimiser l'allocation des ressources. Face à ces enjeux, il est impératif de concevoir des stratégies dynamiques capables d'exploiter efficacement les ressources hétérogènes et instables, garantissant ainsi des performances robustes et une terminaison fiable des tâches.

Dans ce contexte, l'allocation des ressources constitue un enjeu central. Si l'ordonnancement en représente l'aspect le plus critique car il vise à optimiser l'utilisation, réduire les échecs d'exécution et limiter les transferts réseau, d'autres approches d'allocation sont également explorées dans ce travail, afin de tirer pleinement parti des ressources hétérogènes et volatiles mises à disposition par les volontaires. Les méthodes traditionnelles d'ordonnancement présentent plusieurs limites dans le contexte du calcul volontaire. On distingue principalement deux catégories :

- **Les approches naïves** : ces méthodes reposent sur des règles simples, sans réelle prise en compte de l'hétérogénéité ou de la dynamique du système. On cite, la politique *First-Come, First-Served (FCFS)* consiste à exécuter les tâches dans l'ordre de leur arrivée, tandis que l'assignation aléatoire distribue les tâches de manière stochastique entre les volontaires. Bien que faciles à mettre en œuvre, ces approches entraînent souvent une mauvaise utilisation des ressources et une augmentation du temps d'exécution global.
- **Les approches basées sur la connaissance** : ces méthodes cherchent à exploiter



des informations disponibles pour améliorer l’ordonnancement. Parmi elles, *Minimum Execution Time (MET)* assigne chaque tâche au volontaire supposé l’exécuter le plus rapidement, *Weighted Round Robin (WRR)* répartit les tâches proportionnellement aux capacités estimées des volontaires, et les modèles probabilistes tentent d’anticiper les défaillances ou performances futures. Toutefois, ces techniques restent limitées face à la forte volatilité et à l’incertitude propres au calcul volontaire.

Ainsi, malgré leur diversité, les approches classiques peinent à s’adapter aux environnements dynamiques et hétérogènes du calcul volontaire. Cela se traduit par des inefficacités notables : dépassement des délais d’exécution, taux d’échec élevés et utilisation sous-optimale des ressources.

Or, l’apprentissage par renforcement a récemment démontré son efficacité pour l’ordonnancement dans des systèmes distribués tels que le *Volunteer Edge Computing (VEC)*, qui présentent plusieurs caractéristiques proches du calcul volontaire : hétérogénéité, volatilité et contraintes réseau. Ces résultats ouvrent la voie à son application dans notre contexte.

La **question de recherche** que nous posons est donc la suivante :

Comment concevoir une stratégie d’allocation de ressources adaptative, basée sur l’apprentissage par renforcement, capable de tirer parti des ressources hétérogènes et volatiles du calcul volontaire, tout en optimisant les performances globales du système ?

Pour y répondre, nous proposons de modéliser le problème comme un **Processus de Décision Markovien (MDP)**, puis d’implémenter l’algorithme **Asynchronous Advantage Actor-Critic (A3C)**. L’agent est entraîné dans des environnements simulés intégrant des profils réalistes de volontaires et de workflows, puis l’évaluation de sa capacité à réduire le temps d’exécution et les échecs, à améliorer l’utilisation des volontaires, à limiter les coûts de communication et à garantir la scalabilité du système sera faite dans un système réel.

Le plan de ce mémoire est organisé comme suit :

- **Chapitre 1 : État de l’art** – Présente du paradigme du calcul volontaire, des défis liés à l’allocation de ressources, et analyse critique des approches existantes, tant classiques que récentes, y compris celles basées sur le RL.
- **Chapitre 2 : Proposition** – fait la formalisation du problème comme un MDP, description détaillée de l’architecture proposée et intégration de l’algorithme A3C.
- **Chapitre 3 : Expérimentations et résultats** – Mise en œuvre dans un environnement réel, description des scénarios de test, analyse comparative avec des méthodes classiques, et discussion des résultats obtenus.
- **Conclusion et perspectives** – Synthèse des contributions, limites identifiées, et pistes de recherche futures.



Etat de l'art

Sommaire

Introduction	3
1.1 Généralités sur le calcul sur machines volontaires	3
1.1.1 Définition et objectifs	4
1.1.2 Architectures	4
1.1.3 Stratégies pour l'amélioration des performances	5
1.2 Méthodes d'allocation des ressources dans le calcul sur machines volontaires	6
1.2.1 Définition et rôle de l'ordonnancement	7
1.2.2 Méthodes d'allocation des ressources	7
1.2.3 Tableau récapitulatif et comparatif	10
1.2.4 Limites des méthodes traditionnelles	10
1.3 Apprentissage par renforcement pour un ordonnancement adaptatif	11
1.3.1 Présentation du RL pour l'ordonnancement	11
1.3.2 Modélisation du problème d'ordonnancement comme un MDP .	12
1.3.3 Algorithmes de RL	12
1.3.4 Application de A3C dans le volunteer edge-cloud computing .	13
1.3.5 Synthèse	14
Conclusion	15

Introduction

Le calcul sur machines volontaires exploite les ressources inactives d'ordinateurs distribués pour répondre aux besoins croissants en puissance de calcul. Son efficacité repose sur une allocation optimale des ressources, condition essentielle pour éviter retards et pertes d'énergie. Ce chapitre examine ses principes, architectures et stratégies d'allocation, ainsi que les défis liés à l'hétérogénéité et à la volatilité.

1.1 Généralités sur le calcul sur machines volontaires

Le travail que nous présentons s'articule sur les systèmes de calcul sur machines volontaires qui sont des systèmes qui utilisent le paradigme de calcul volontaire ou *Volunteer Computing* (VC) pour offrir des ressources de calcul élevées. Pour cela, il est impératif de

présenter ce paradigme, les architectures de ses systèmes et les stratégies utilisées pour améliorer leurs performances.

1.1.1 Définition et objectifs

Le calcul sur machines volontaires est un type de calcul distribué dans lequel toute personne disposant d'un ordinateur peut faire don de ses ressources informatiques inutilisées pour exécuter des tâches nécessitant une puissance de calcul et un espace de stockage importants [2]. Il s'inscrit dans le cadre d'efforts continus visant à trouver une infrastructure informatique alternative moins coûteuse pour les applications très gourmandes en ressources[1]. Ses objectifs sont multiples : offrir une alternative économique et écologique aux superordinateurs, mobiliser des ressources sous-exploitées (souvent inférieures à 10 % d'utilisation CPU dans les environnements domestiques [3]), et accélérer des avancées dans des domaines comme la recherche scientifique, médicale ou environnementale. Ce paradigme convertit l'inactivité des ressources en une opportunité collaborative au service de l'innovation scientifique. Pour comprendre ce type de système, il est impératif de comprendre ses différentes architecture, nous les présentons dans la section suivante

1.1.2 Architectures

L'architecture d'un système informatique définit l'organisation de ses composants et leurs interactions, déterminant ainsi pour les systèmes de calcul sur machines volontaires la manière dont les tâches sont distribuées, les ressources exploitées et les résultats collectés. Dans le calcul sur machines volontaires, elle joue un rôle clé dans la gestion des ordinateurs personnels pour assurer une exécution efficace des calculs distribués.

Ces systèmes reposent sur trois composantes principales, dont les rôles varient selon l'architecture mais restent essentiels :

- **Le serveur** : joue un rôle central en assurant l'enregistrement des volontaires et des clients, ainsi que la mise à jour de la liste des participants. Selon la configuration du système, il peut également recevoir les tâches de calcul, les répartir entre les volontaires disponibles, puis collecter et agréger les résultats obtenus. Dans d'autres cas, sa fonction se limite à maintenir à jour la base d'informations sur les volontaires et à gérer les statistiques des calculs réalisés.
- **Les volontaires** : exécutent les tâches qui leur sont assignées durant leurs périodes d'inactivité, pendant le temps défini par leur propriétaire, ou encore lorsque l'utilisation de la ressource reste en dessous d'un seuil fixé. Une fois l'exécution achevée, les résultats sont retournées au système.
- **Les clients** : représentent les entités qui soumettent les tâches à traiter. Il peut s'agir de chercheurs, d'entreprises ou de particuliers ayant besoin d'une puissance de calcul supplémentaire.

Ces rôles influencent directement la gestion des ressources, en déterminant comment les tâches sont assignées, exécutées et les résultats collectés [1]. Du point de vue systémique, trois architectures principales se distinguent, chacune impactant l'allocation des ressources différemment :

Architecture centralisée ou client/serveur Dans ce modèle, une machine ou un ensemble de machines dédiées agit comme serveur et coordonne toutes les ressources du système [1, 4]. Il reçoit les calculs soumis par les clients, les décompose en tâches élémentaires, sélectionne les volontaires disponibles et leur attribue les calculs [4, 5]. Il assure également la collecte et la validation des résultats. Cette architecture simplifie

la supervision, mais elle impose également une forte vulnérabilité au point unique de défaillance, car la surcharge du serveur peut compromettre l'efficacité de ses décisions de gestion, tandis qu'une gestion inadéquate des critères de fiabilité et de disponibilité peut mener à des délais non respectés ou à des résultats invalides. L'impact de ce modèle est donc double : il permet une vision globale et un contrôle cohérent de l'allocation, mais rend la gestion des ressources et l'ordonnancement sensibles à la performance et à la robustesse du serveur central.

Architecture décentralisée ou pair à pair Dans une organisation décentralisée, la coordination ne repose pas sur un serveur unique, mais est répartie entre plusieurs nœuds coopérants. Chaque nœud peut assurer partiellement la distribution des tâches, la collecte des résultats ou le suivi des volontaires. Cette approche améliore la tolérance aux pannes et la scalabilité en répartissant les responsabilités sur plusieurs entités [6]. Toutefois, l'absence de vision globale complique l'allocation des ressources et l'ordonnancement, qui doivent reposer sur des mécanismes distribués (protocoles pair-à-pair, consensus ou systèmes multi-agents). Les décisions locales peuvent alors être sous-optimales, entraînant parfois une surcharge de certains volontaires et une sous-utilisation d'autres.

Architecture hybride. L'architecture hybride combine les avantages des approches centralisée et décentralisée. Un serveur ou un ensemble restreint de serveurs assurent la gestion générale des volontaires, l'enregistrement des volontaires et la distribution initiale des tâches, tandis que certaines décisions locales d'ordonnancement ou de partage de résultats sont déléguées aux volontaires eux-mêmes ou à des sous-serveurs [5]. Ce modèle cherche à équilibrer la cohérence de la centralisation et la résilience de la décentralisation. Il permet une meilleure répartition de la charge et une plus grande tolérance aux pannes, tout en maintenant un niveau raisonnable de supervision pour garantir la fiabilité des résultats.

Les architectures présentées, qu'elles soient centralisées, décentralisées ou hybrides, mettent en évidence l'importance stratégique de l'allocation des ressources dans les systèmes de calcul sur machines volontaires. En effet, la performance et la fiabilité d'un tel système dépendent largement de la manière dont les tâches sont réparties entre des volontaires. L'allocation des ressources et l'ordonnancement des tâches constituent donc un problème central dont le but est de garantir la terminaison des tâches dans les meilleurs délais. Dans les systèmes existants, plusieurs approches ont été utilisées pour y répondre, allant d'heuristiques simples à des techniques d'optimisation plus élaborées, afin d'améliorer le temps d'exécution, l'équilibre de charge, la tolérance aux pannes et la qualité des résultats. Dans ce qui suit, nous les présentons.

1.1.3 Stratégies pour l'amélioration des performances

L'environnement dynamique des systèmes Volunteer Edge-Cloud (VEC) entraîne des fluctuations de performance, provoquant des retards d'exécution et une diminution de l'efficacité. Pour remédier à ces problèmes, deux stratégies principales sont utilisées : la migration et la réplication des tâches, qui visent à renforcer la robustesse du système face à la volatilité et à l'hétérogénéité des nœuds.

1.1.3.1 Migration des tâches

La migration consiste à transférer une tâche en cours d'exécution d'un nœud volontaire (VN) vers un autre en cas de défaillance, comme une déconnexion ou une surcharge



des ressources, permettant ainsi de garantir la continuité de l'exécution sans repartir de zéro [7]. Ce processus repose sur des captures périodiques de l'état de la tâche, envoyées au serveur central pour stockage et transfert éventuel. Bien que cette approche assure l'achèvement des tâches, elle engendre des coûts supplémentaires, notamment une augmentation de l'espace de stockage requis sur le serveur et les nœuds volontaires, ainsi qu'une charge réseau due aux transferts réguliers des états capturés. Malgré ces contraintes, la migration est essentielle dans les environnements volatils pour minimiser l'impact des interruptions imprévues des nœuds, renforçant ainsi la fiabilité du système.

1.1.3.2 Réplication des tâches

La réplication est une technique couramment employée dans les systèmes de calcul distribué tels que Condor et BOINC [8]. Elle consiste à créer plusieurs copies d'une tâche, appelées instances (dont le nombre varie selon le système), et à les distribuer aux nœuds volontaires pour exécution. Dans BOINC, par exemple, les nœuds sont regroupés en fonction de leur système d'exploitation et de l'architecture de leur processeur, de sorte que les instances d'une tâche sont envoyées uniquement aux nœuds appartenant à la même catégorie [8]. Les nœuds exécutant ces instances peuvent être sélectionnés de manière aléatoire ou en fonction de leur puissance de calcul parmi les disponibles, afin de répartir équitablement les tâches et d'éviter une surcharge sur certains nœuds [8]. Parmi ses avantages, la réplication améliore la fiabilité des résultats grâce au mécanisme de quorum, où un consensus parmi les répliques assure des résultats précis malgré les défaillances individuelles, tout en réduisant potentiellement le temps d'exécution global en distribuant les instances sur plusieurs nœuds [8]. Cependant, elle présente des inconvénients notables, tels qu'un risque de gaspillage de ressources lorsque plusieurs nœuds travaillent simultanément sur des copies identiques, une augmentation de la charge réseau due à la communication fréquente des états d'exécution au serveur central, et une dépendance à la disponibilité et à la performance individuelle de chaque nœud, ce qui peut affecter la cohérence des résultats et la durée totale d'exécution.

Bien que la migration et la réplication offrent des mécanismes efficaces pour gérer les défaillances et les incertitudes, leur efficacité dépend largement d'une assignation initiale intelligente des tâches. C'est pourquoi ces stratégies sont souvent intégrées à des approches d'ordonnancement avancées, qui prennent en compte des critères comme la fiabilité des nœuds et leur historique pour optimiser les performances globales du système. Cette combinaison pave la voie à des méthodes plus sophistiquées d'allocation des ressources, telles que celles explorées dans la section suivante.

1.2 Méthodes d'allocation des ressources dans le calcul sur machines volontaires

Les machines volontaires constituent un environnement hétérogène et instable, où les différences de capacités, de disponibilité et de fiabilité compliquent l'exploitation directe des ressources. L'ordonnancement agit comme un mécanisme d'organisation, coordonnant





ces ressources pour maximiser les performances. Cette section présente l'ordonnancement, examine les méthodes d'allocation des ressources par ordre croissant de complexité et souligne les limites des approches traditionnelles face ces défis.

1.2.1 Définition et rôle de l'ordonnancement

L'ordonnancement dans le calcul sur machines volontaires consiste à assigner des tâches aux volontaires de manière à maximiser les performances globales du système tout en respectant les contraintes de temps, de fiabilité et de ressources [1]. Ce processus intervient à deux niveaux : côté serveur, où des décisions globales sont prises pour distribuer les tâches, et côté volontaire, où la gestion locale, comme l'utilisation de tampons de tâches, optimise l'exécution [1]. Les ressources à ordonnancer incluent les volontaires, selon leurs capacités (CPU, mémoire, bande passante) à répondre aux exigences des tâches, et les tâches elles-mêmes, qui doivent être assignées ou reportées en fonction des contraintes énoncées pour leur exécution.

Dans un système comme BOINC, par exemple, un serveur central décide quelles tâches attribuer à des milliers de volontaires, tandis que chaque volontaire gère localement un tampon de tâches pour minimiser les interactions réseau [4]. Cette dualité serveur-volontaire est essentielle pour gérer l'hétérogénéité des appareils et leur volatilité. L'ordonnancement efficace est donc le pivot qui transforme des ressources dispersées en une puissance de calcul cohérente, mais il exige des stratégies adaptées pour surmonter les défis du VC.

Avec cette définition posée, examinons les stratégies d'allocation des ressources, en progressant de la simplicité des méthodes naïves à la sophistication des approches basées sur la connaissance.

1.2.2 Méthodes d'allocation des ressources

Les méthodes d'allocation des ressources dans le VC tentent de dompter le chaos inhérent à un réseau de volontaires aux capacités variées et à la disponibilité imprévisible. La plupart d'entre elles sont héritées des méthodes d'ordonnancement dans les systèmes distribués, pendant que les autres sont propres aux VC. Elles se divisent en deux classes : les méthodes naïves, qui ignorent l'historique des volontaires (FCFS, assignation aléatoire, localité), et les méthodes basées sur la connaissance, qui exploitent l'historique d'exécution (MET, Roun Robin Basé sur un Seuil, Tampon N , WRR, probabilistes et évolutionnaires) [1].

1.2.2.1 First-Come-First-Served (FCFS)

La méthode First-Come-First-Served (FCFS) assigne chaque tâche au premier volontaire disponible, suivant une file FIFO (First In, First Out) gérée par le serveur [6]. Sa simplicité en fait un choix naturel pour les systèmes à faible échelle avec des participants aux capacités presque similaires et une charge stable. Par exemple, dans un projet de calcul volontaire précoce comme SETI@home dans sa première phase, FCFS permettait une distribution rapide des tâches d'analyse de signaux radio sans analyse préalable des capacités des nœuds [4].

Cependant, cette approche ignore l'hétérogénéité des volontaires. Une tâche computationnelle lourde peut être assignée à un nœud lent, bloquant la file et augmentant les délais [6]. De plus, le fait d'ignorer l'historique d'exécution peut entraîner l'attribution de la tâche à un nœud lent par le passé [1]. Ces limitations conduisent à explorer l'assignation aléatoire pour mieux répartir la charge.





1.2.2.2 Assignation aléatoire

L'assignation aléatoire sélectionne un volontaire disponible au hasard, sans considération de ses performances ou de son historique [1]. Gérée côté serveur, cette méthode convient aux systèmes avec un grand nombre de nœuds presque homogènes, où la probabilité de choisir un nœud performant est élevée. Dans un système de calcul volontaire à grande échelle, cette approche évite le blocage systématique observé avec FCFS en répartissant les tâches de manière plus rapide [2].

Néanmoins, l'absence d'optimisation basée sur les capacités des nœuds peut conduire à des assignations inefficaces, ce qui a pour conséquences la fluctuation des performances du système dans le temps. Pour réduire les transferts réseau, l'ordonnancement basé sur la localité offre une alternative en priorisant les nœuds disposant déjà des données nécessaires au calcul.

1.2.2.3 Ordonnancement basé sur la localité

L'ordonnancement basé sur la localité assigne les tâches aux nœuds détenant déjà les données requises, minimisant ainsi les transferts réseau [4]. Cette méthode, gérée côté serveur, est particulièrement adaptée aux applications manipulant de grands volumes de données, comme le cas de analyses des protéines avec Rosetta@home dans BOINC [2].

Cependant, cette approche présente plusieurs limites. Le nœud disposant des données nécessaires n'est pas toujours le plus performant ni le mieux placé en termes de disponibilité, ce qui peut conduire à un ralentissement global du traitement. De plus, en favorisant systématiquement la proximité des données, le planificateur risque de créer des déséquilibres de charge, certaines machines devenant saturées tandis que d'autres demeurent sous-exploitées. Dans le cas où les données sont très demandées, des points de concentration apparaissent, augmentant le risque de congestion et allongeant les délais d'exécution. Enfin, lorsque les ensembles de données évoluent, maintenir leur cohérence entre différentes copies locales engendre un surcoût supplémentaire en termes de gestion et de communication.

La nécessité d'une pré-réplication des données impose également un coût de stockage non négligeable, limitant l'applicabilité de cette méthode dans des environnements contraints [2]. Pour exploiter plus efficacement l'historique des performances des volontaires et dépasser ces limites, la méthode du délai d'exécution moyen (MET) constitue une alternative plus flexible.

1.2.2.4 Temps d'exécution moyen (MET)

La méthode Mean Execution Time (MET) s'appuie sur l'analyse de l'historique des temps d'exécution afin de sélectionner préférentiellement les nœuds présentant les durées moyennes d'exécution les plus faibles [4]. Gérée côté serveur, elle se révèle particulièrement adaptée aux environnements hétérogènes tels que BOINC, où l'attribution de tâches lourdes à des machines performantes améliore sensiblement l'efficacité.

Toutefois, cette approche présente certaines limites. Les données historiques sur lesquelles elle repose peuvent devenir rapidement obsolètes, ce qui compromet la pertinence des décisions dans des environnements dynamiques. Par ailleurs, en concentrant systématiquement les affectations sur les nœuds les plus rapides, elle tend également à créer un déséquilibre de charge, tout en négligeant des aspects essentiels tels que la fiabilité ou le taux d'erreur des volontaires. Enfin, en se fondant exclusivement sur les performances passées, MET limite l'intégration de nouvelles ressources et réduit la capacité d'adaptation du système.





Pour atténuer ces inconvénients, plusieurs variantes ont été proposées, combinant MET avec des seuils de fiabilité ou des mécanismes de rotation, afin de diversifier l'allocation et d'améliorer la robustesse des décisions d'ordonnancement.

1.2.2.5 Tourniquet et priorité basée sur un seuil

Cette méthode repose sur l'établissement de seuils de fiabilité et de disponibilité que les nœuds doivent satisfaire pour être éligibles à l'exécution, combinés à une distribution des tâches selon un mécanisme de type tourniquet (Round Robin) [2]. Entièrement gérée côté serveur, elle est particulièrement adaptée aux projets scientifiques critiques où la fiabilité prime sur l'exploitation exhaustive des ressources.

Toutefois, l'approche présente plusieurs limites. En excluant systématiquement les nœuds ne respectant pas les seuils, elle réduit l'utilisation globale du parc disponible et introduit une rigidité qui ne s'adapte pas aux fluctuations de disponibilité ou de performance des volontaires. De plus, la gestion centralisée de la sélection et de la rotation accroît la charge du serveur, tout en conservant un point de défaillance unique. Enfin, si le tourniquet favorise une certaine équité dans la répartition, il ne tient pas compte de l'hétérogénéité des performances, ce qui peut ralentir l'exécution de workflows (ensemble de tâches interdépendantes ou non) complexes.

Pour atténuer ces contraintes, la stratégie du tampon N a été proposée pour réduire la dépendance au serveur central et améliorer la tolérance aux interruptions réseau.

1.2.2.6 Stratégie du tampon N

La stratégie du tampon N consiste à précharger un ensemble de N tâches de taille déterminée sur chaque volontaire, qui les exécute localement sans interagir avec le serveur tant que son tampon n'est pas épuisé [1]. Le serveur conserve uniquement le rôle de définir la taille initiale, tandis que l'exécution est assurée côté volontaire. Cette approche est particulièrement adaptée aux environnements où la connectivité réseau est faible ou intermittente. Elle a été largement mise en œuvre dans BOINC, où elle permet aux volontaires de continuer à travailler en mode déconnecté tout en exploitant leur autonomie locale [4].

Toutefois, le choix de la taille du tampon constitue un paramètre critique. Lorsqu'il est trop important, il surcharge les volontaires aux capacités limitées; inversement, un tampon trop réduit entraîne une multiplication des interactions réseau, ce qui diminue l'efficacité globale. L'absence d'adaptation dynamique aux conditions du système accentue cette rigidité, et un mauvais réglage peut accroître le temps d'attente du workflow de manière globale[1]. Ces limites ont conduit à l'émergence d'approches hybrides(côté client et côté serveur), telles que le *Weighted Round Robin* (WRR), qui introduisent une gestion plus fine des priorités et tiennent compte de l'hétérogénéité des nœuds.

1.2.2.7 Weighted Round Robin (WRR)

Le *Weighted Round Robin* (WRR) attribue des poids aux projets ou aux tâches au sein de la file locale d'un volontaire, qui sont ensuite exécutées selon un cycle proportionnel à ces poids [1]. Utilisé dans BOINC, ce mécanisme permet de gérer la concurrence entre projets et d'assurer une répartition plus équilibrée des ressources. Néanmoins, son efficacité reste fortement dépendante de la configuration des poids. Ces poids sont généralement statiques et ne tiennent pas compte de la volatilité des hôtes, de leur performance effective ni des variations de charge réseau. De plus, WRR peut entraîner une sous-utilisation des ressources lorsque des poids élevés sont attribués à des projets disposant de peu de tâches,





laissant certaines capacités des volontaires inutilisées. Par ailleurs, il peut provoquer une augmentation des délais d'exécution : les projets faiblement pondérés voient leurs tâches retardées, et la déconnexion prématurée des volontaires accentue ces retards en nécessitant une réassignation des tâches. Ces limites ont motivé l'exploration de méthodes plus flexibles, notamment les approches probabilistes et évolutionnaires.

1.2.2.8 Méthodes probabilistes et évolutionnaires

Les méthodes probabilistes et évolutionnaires reposent sur des modèles statistiques et des algorithmes inspirés de l'évolution, tels que les algorithmes génétiques, afin de prédire la disponibilité et la performance des nœuds [9]. Certaines variantes, comme le look-ahead genetic algorithm (LAGA), intègrent des mécanismes de réputation temporelle afin de renforcer la fiabilité de l'ordonnancement dans les applications de workflow [9]. D'autres approches recourent à des modèles stochastiques comme le processus de Poisson pour modéliser le taux de requêtes de tâches en fonction de la période de reconnexion des volontaires [9]. Ces techniques combinent généralement un ordonnancement côté serveur, chargé de sélectionner les nœuds les plus adaptés, et une exécution locale côté volontaire, ce qui en fait une solution hybride adaptée aux environnements volatils. Toutefois, leur principal inconvénient réside dans leur complexité computationnelle : la construction de modèles prédictifs ou l'exécution d'algorithmes évolutionnaires induisent une surcharge serveur importante, qui compromet leur scalabilité dans des systèmes à grande échelle. De plus, la convergence parfois lente de ces méthodes peut retarder la prise de décision, ce qui limite leur applicabilité pratique.

1.2.3 Tableau récapitulatif et comparatif

Le tableau 1.1 synthétise les méthodes selon les critères de performance, fiabilité, équité, localité, complexité, évolutivité et adaptabilité, offrant une vue d'ensemble claire pour comparer leurs forces et faiblesses [1].

TABLE 1.1 – Comparaison des méthodes d'allocation de ressources dans le calcul volontaire

Méthode	Performance	Fiabilité	Équité	Localité	Complexité	Évolutivité	Adaptabilité
FCFS	Faible à moyenne	Moyenne	Faible	Nulle	Très faible	Élevée	Faible
Assignation aléatoire	Moyenne	Moyenne	Équitable	Nulle	Très faible	Élevée	Faible
Ordonnancement basé sur la localité	Élevée	Moyenne	Moyenne	Très élevée	Faible à moyenne	Moyenne à élevée	Faible
Délai d'exécution moyen	Élevée	Variable	Moyenne	Faible	Moyenne	Moyenne	Moyenne
Tourniquet et priorité basée sur un seuil	Moyenne à élevée	Élevée	Moyenne	Faible	Moyenne	Moyenne	Moyenne
Stratégie du tampon N	Moyenne à élevée	Élevée	Moyenne	Moyenne	Moyenne	Moyenne	Élevée
Weighted Round Robin	Moyenne	Moyenne à élevée	Pondérée	Faible	Moyenne à élevée	Moyenne	Moyenne
Méthodes probabilistes et évolutionnaires	Élevée	Élevée	Élevée	Moyenne	Élevée	Moyenne	Très élevée

1.2.4 Limites des méthodes traditionnelles

En dépit de leurs apports, les méthodes classiques d'allocation et d'ordonnancement demeurent insuffisantes pour exploiter de manière optimale les ressources des systèmes de calcul volontaire, en raison de la volatilité et de l'hétérogénéité des nœuds participants. Leur efficacité se trouve ainsi réduite dans des environnements contraints, et leur dépendance à des hypothèses statiques ou à des modèles simplifiés limite leur pertinence dans des contextes dynamiques. Ces insuffisances se manifestent principalement à travers les points suivants :

- **Difficulté à gérer la volatilité des nœuds** : Les méthodes telles que le tampon N ou le WRR échouent à maintenir des performances stables lorsque les taux de





déconnexion atteignent des niveaux élevés, entraînant une baisse significative du taux de succès des tâches [1].

- **Incapacité à exploiter pleinement l'hétérogénéité** : L'assignation uniforme ou basée sur des règles statiques ne tient pas compte des écarts de performance entre appareils, provoquant des retards et une utilisation sous-optimale des ressources disponibles [2].
- **Rigidité des modèles de décision** : Les approches déterministes, comme le WRR, reposent sur des paramètres fixés a priori (poids, taille du tampon) qui s'avèrent inefficaces dans un environnement dont la charge évolue en permanence.
- **Surcharge computationnelle des approches avancées** : Les méthodes probabilistes et évolutionnaires, bien qu'adaptatives, introduisent une complexité de calcul importante côté serveur, ce qui freine leur déploiement dans des systèmes de grande échelle [9].
- **Taux d'échec accrus dans des environnements dynamiques** : L'utilisation de données historiques obsolètes ou de règles statiques conduit à une augmentation des échecs de tâches, compromettant la fiabilité globale [2].

Ces limites soulignent la nécessité d'approches plus intelligentes, capables de concilier adaptativité, efficacité et scalabilité. Contrairement aux méthodes traditionnelles, qui reposent sur des règles statiques et manquent de flexibilité face aux environnements dynamiques, les techniques d'intelligence artificielle, et en particulier l'apprentissage par renforcement.

1.3 Apprentissage par renforcement pour un ordonnancement adaptatif

L'émergence de l'intelligence artificielle a révolutionné de nombreux domaines, de la reconnaissance d'images à la prévision financière, en offrant des solutions adaptatives et performantes. Parmi ses branches, l'apprentissage par renforcement ou Reinforcement Learning (RL) trouve une application croissante dans les systèmes distribués, particulièrement pour l'ordonnancement des tâches, où il permet d'optimiser les décisions séquentielles face à des environnements dynamiques. Le RL permet de concevoir des politiques adaptatives capables de répondre aux fluctuations des ressources et des charges de travail, surpassant souvent les approches traditionnelles dans des contextes dynamiques et hétérogènes [10].

1.3.1 Présentation du RL pour l'ordonnancement

L'apprentissage par renforcement (RL) est une branche de l'apprentissage automatique où un agent apprend à prendre des décisions optimales en interagissant avec un environnement [11, 12]. Ses principaux composants sont :

- **L'agent** : entité décisionnelle qui, dans un système distribué par exemple, correspond à l'ordonnanceur chargé d'assigner les tâches aux ressources ;
- **L'environnement** : le système dans lequel sont appliquées les décisions de l'agent ;
- **Les états** : représentations de l'observation de l'environnement à un instant donné ;
- **Les actions** : liste des décisions que l'agent peut prendre et appliquer sur l'environnement ;
- **Les récompenses** : signaux évaluant la qualité des actions, ajustés selon les objectifs de l'agent.





À chaque étape, l'agent observe l'état courant de l'environnement, sélectionne une action, l'environnement évolue vers un nouvel état, puis l'agent reçoit une récompense (ou une pénalité), et le cycle recommence.[12]. L'objectif est d'apprendre une *politique* ou une stratégie qui associe chaque état à une action maximisant la somme cumulée des récompenses à long terme. Contrairement aux approches supervisées, le RL ne nécessite pas de données étiquetées, mais repose sur l'exploration et l'exploitation des retours de l'environnement.

Pour les environnements complexes avec de grands espaces d'états ou d'actions, le *Deep RL* (DRL) combine le RL avec des réseaux de neurones profonds pour approximer les politiques ou les fonctions de valeur [11, 12]. Le RL est particulièrement adapté aux problèmes de prise de décision séquentielle, comme l'optimisation de processus dynamiques, et repose sur un cadre formel appelé processus de décision markovien (MDP).

1.3.2 Modélisation du problème d'ordonnancement comme un MDP

L'ordonnancement des tâches dans les systèmes distribués est un problème complexe, caractérisé par l'hétérogénéité des ressources et la variabilité des charges. Sa modélisation en MDP permet de formaliser ces dynamiques via un cadre mathématique rigoureux [13]. Un MDP est défini par le tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, adapté comme suit :

- $\mathcal{S}$: ensemble des états, incluant les caractéristiques et l'état des tâches (taille, priorité, dépendances), l'état des ressources (CPU, mémoire, machines virtuelles, etc), et les files d'attente.
- $\mathcal{A}$: ensemble des actions, comme assigner une tâche t_i à une ressource $v_j \in V$ ou reporter l'assignation (NULL). V étant l'ensemble des nœuds du système.
- $T(s'|s, a)$: probabilités de transition, modélisant la probabilité de passer à l'état s' lorsqu'on a pris l'action a étant à l'état s .
- $R(s, a)$: récompense, définie comme la combinaison de plusieurs objectifs selon l'action a prise à l'état s , $R(s, a) = w_1 \cdot R_{\text{makespan}} + w_2 \cdot R_{\text{utilisation}} + w_3 \cdot R_{\text{coût}}$, où les poids w_k équilibrent les objectifs [11].
- $\gamma \in [0, 1]$: facteur d'actualisation encore appelé facteur d'escompte pour pondérer les récompenses futures [12].

Cette formalisation permet d'optimiser le makespan, d'équilibrer la charge et de gérer l'incertitude inhérente aux environnements distribués. Les approches DRL, en particulier, surmontent la complexité liée à la grande dimension des espaces d'états et d'actions. Plusieurs algorithmes sont utilisés pour résoudre les MDP, nous les présentons dans la section suivante.

1.3.3 Algorithmes de RL

Les algorithmes de RL appliqués à l'ordonnancement dans les systèmes distribués se classent en quatre catégories principales : les méthodes sans modèle, les méthodes basées sur la politique, les approches multi-agents et les approches avancées. Chacune est adaptée à des contextes spécifiques, avec des performances mesurées dans la littérature [13]. Nous distinguons :

Méthodes sans modèle : ces algorithmes apprennent directement à partir des interactions sans avoir modélisé l'environnement, idéales pour les systèmes cloud dynamiques [10]. Elles se résument à la construction d'une table nommée Q-table qui associe chaque action dans un état à une récompense. Ces algorithmes sont :





- **DQN (Deep Q-Network)** : utilise un réseau neuronal pour approximer la fonction de valeur $Q(s, a)$, estimant le gain cumulé d'une action dans un état. DQN est efficace pour les espaces d'actions discrets, comme l'assignation de tâches à des VM.
- **DDQN (Double DQN)** : améliore DQN en découplant la sélection et l'évaluation des actions, réduisant les biais de surestimation.
- **Dueling DQN** : sépare la valeur de l'état et l'avantage de chaque action, améliorant l'efficacité lorsque certaines actions ont un impact limité.
- **Noisy DQN** : intègre du bruit appris dans les poids du réseau pour favoriser une exploration robuste.

Méthodes à base de politique : ces méthodes apprennent une politique $\pi(a|s)$ directement sans conserver une correspondance entre les actions, états et récompenses ; adaptée aux espaces d'actions continus ou complexes.

- **A2C/A3C (Actor-Critic)** : combine un acteur qui prend des décisions et un critique qui évalue la politique. A3C, en particulier, utilise un apprentissage asynchrone pour accélérer la conver
- **PPO (Proximal Policy Optimization)** : utilise des mises à jour clipées pour stabiliser l'apprentissage, performant pour l'ordonnancement en temps réel. Cela lui donne le contrôle la mise à jour de la politique par une contrainte sur la distance avec l'ancienne politique, offrant stabilité et efficacité.
- **DDPG (Deep Deterministic Policy Gradient)** : conçu pour les actions continues, avec exploration par bruit additif.
- **TD3 (Twin Delayed DDPG)** : amélioration de DDPG avec double Q-network et mises à jour retardées, offrant une stabilité accrue.

Méthodes multi-agents : gèrent plusieurs agents coopérant pour optimiser l'ordonnancement.

- **MADDPG (Multi-Agent Deep Deterministic Policy Gradient)** : étend DDPG aux environnements multi-agents, avec un entraînement centralisé et une exécution décentralisée.
- **MAPPO (Multi-Agent Proximal Policy Optimization)** : adapte PPO pour des scénarios coopératifs, offrant une stabilité dans les environnements complexes.
- **MA3C (Multi-Agent Asynchronous Advantage Actor-Critic)** : combine apprentissage asynchrone et actor-critic pour une convergence rapide.

Méthodes avancées : hybrides ou spécialisées pour des environnements complexes.

- **Hybrides heuristique-DRL** : intègrent des heuristiques pour initialiser les politiques ou guider l'exploration, réduisant le temps d'apprentissage.
- **QRL (Quantum Reinforcement Learning)** : exploite des circuits quantiques pour explorer plus efficacement l'espace des états.

1.3.4 Application de A3C dans le volunteer edge-cloud computing

Le volunteer edge-cloud (VEC) repose sur des nœuds volontaires (VNs) variés, allant des dispositifs IoT aux serveurs, avec des capacités hétérogènes et une disponibilité intermittente [15]. L'ordonnancement dans le VEC vise à optimiser des workflows scientifiques, comme PGen (analyse génomique) ou RNA-Seq (expression génique), sous contraintes de QSpecs (Quality Of Service Specifications) et SSspecs (Security Specifications) [15]. Une étude clé utilise A3C pour modéliser l'ordonnancement comme un MDP, avec des états





TABLE 1.2 – Synthèse des approches DRL pour l’ordonnancement des tâches dans le cloud

Méthode	Type	Avantages	Limitations
DQN	Basée sur la valeur	Robustesse, réduction makespan [14]	Surestimation Q
DDQN	Basée sur la valeur	Stabilité, erreurs réduites [13]	Complexité computationnelle
PPO	Basée sur la politique	Stabilité, réduction coûts [13]	Dépendance hyperparamètres
A3C	Acteur-critique	Efficacité workflows [13]	Surcharge grande échelle
MADDPG	Multi-agents	Utilisation ressources [13]	Complexité coordination
Hybride DRL-Heuristique	Hybride	Précision accrue [13]	Dépendance heuristiques
QRL	Avancée	Exploration efficace [13]	Accès infrastructures quantiques

incluant les caractéristiques des tâches (taille, dépendances), l’état des files (quantifié en quatre niveaux : faible, moyen, élevé, complet), et les préférences des VNs (basées sur la réputation des utilisateurs) [15]. Les actions consistent à assigner une tâche à un VN ou à la rejeter, avec une récompense combinant satisfaction des QSpecs/SSpecs, préférences des VNs, et fiabilité [15].

Les résultats, évalués sur Kubernetes Nautilus avec des workflows réels et synthétiques, montrent qu’A3C atteint 96.55 % de satisfaction QSpecs et 98.27 % pour SSpecs à faible taux d’arrivée ($\lambda = 0.02 - 0.04$), surpassant les baselines Random et GR [15]. Par rapport à VECFlex (basé sur PSO), A3C réduit les rejets de tâches allant jusqu’à 20 % pour les workflows PGen, grâce à une meilleure gestion des dépendances DAG [15]. L’utilisation des VNs atteint 50-70 % même pour les nœuds moins performants, avec un écart-type faible, indiquant une allocation équilibrée [15]. A3C excelle particulièrement pour les workflows complexes (PGen avec 100+ tâches interdépendantes), réduisant le makespan de 25 % comparé à PSO et maintenant une satisfaction SSpecs élevée (98 % vs. 90 % pour PSO) [15]. Ces performances sont attribuées à l’apprentissage asynchrone d’A3C, qui gère l’incertitude des VNs et optimise les assignations en temps réel [16].

1.3.5 Synthèse

Le RL s’adapte en temps réel aux environnements complexes grâce à son processus d’essai-erreur : l’agent explore des actions, observe les récompenses, et ajuste sa politique pour maximiser les gains cumulés à long terme, sans nécessiter un modèle explicite de l’environnement [11]. Cette adaptabilité est cruciale dans les systèmes distribués, où les ressources varient en disponibilité et performance, permettant par exemple de réassigner dynamiquement une tâche à une nouvelle machine virtuelle en cas de panne, réduisant les interruptions [12]. De plus, le RL apprend des interactions passées, assurant une optimisation continue face aux incertitudes comme les fluctuations des charges réseau ou des ressources. Ces avantages font du RL une approche idéale pour l’ordonnancement dans des environnements cloud dynamiques. Les algorithmes RL, comme A3C, démontrent une supériorité dans les systèmes distribués et cloud, réduisant le makespan de 20-30 % et améliorant l’utilisation des ressources face à l’hétérogénéité et la volatilité [10, 13, 15]. Ces résultats sont prometteurs pour VC, où les ressources bénévoles présentent des défis similaires. Ainsi, nous pouvons adapter le RL en s’appuyant sur A3C, pour optimiser l’ordonnancement des tâches dans le VC, en tenant compte de la dynamique unique des ordinateurs personnels.



Conclusion

Ce chapitre a exploré le calcul sur machines volontaires, ses architectures, et les méthodes d'allocation des ressources, mettant en évidence les limites des approches traditionnelles face à l'hétérogénéité et la volatilité. L'apprentissage par renforcement, via des algorithmes comme A3C, offre une solution adaptative, modélisant l'ordonnancement comme un MDP pour optimiser les performances dans des environnements dynamiques. Le chapitre suivant proposera une approche RL pour l'ordonnancement dans le VC, s'inspirant des succès dans le cloud computing.

Approche d’ordonnancement de tâches dans le calcul volontaire utilisant l’apprentissage par renforcement.

Sommaire

Introduction	16
2.1 Modélisation du Problème d’Ordonnancement	17
2.1.1 Définition des Entités et Contraintes du Système	17
2.1.2 Modélisation de la Volatilité et Disponibilité	18
2.1.3 Scores de Satisfaction pour l’Évaluation des Assignations	19
2.2 Solution basée sur Apprentissage par Renforcement	22
2.2.1 Formalisation sous un Processus de Décision Markovien (MDP)	23
2.2.2 Architecture A3C Adaptée	23
2.3 Solution Algorithmique et Perspectives	25
2.3.1 Algorithme d’Apprentissage	25
2.3.2 Synthèse et Avantages de l’Approche	27
Conclusion	27

Introduction

Dans le chapitre précédent, nous avons mis en évidence les limites des méthodes existantes dans la littérature pour s’adapter dynamiquement aux fluctuations des ressources dans les environnements de VC. Nous avons aussi montré potentiel du RL à gérer efficacement les environnements dynamiques pour l’ordonnancement des tâches. Inspirés par les avancées dans le VEC telles que celles présentées par [15], ce chapitre vise à présenter une approche basée sur le RL spécifiquement adaptée à l’ordonnancement des tâches dans les systèmes VC. Il est organisé comme suit : la section 2.1 présente la modélisation du système et les métriques pour l’ordonnancement et pose le problème d’optimisation, la section 2.2 formalise le problème comme un MDP et propose une solution adaptative. Enfin, la section 2.3 expose la solution algorithmique basée sur A3C, accompagnée d’une synthèse des avantages.

2.1 Modélisation du Problème d'Ordonnancement

Un système VC repose sur la soumission de tâches computationnelles par un client à un serveur structuré comme un graphe acyclique dirigé (DAG) pour représenter les dépendances. Le serveur distribue les tâches à des volontaires, qui les exécutent et renvoient les résultats. Ces résultats sont ensuite agrégés et transmis au client. Pour concevoir une stratégie d'ordonnancement efficace de ces tâches parmi les volontaires, une modélisation rigoureuse du système est essentielle. Cette modélisation vise à faciliter une compréhension approfondie des dynamiques du système et à permettre une prise de décision optimale.

2.1.1 Définition des Entités et Contraintes du Système

Dans un système VC, les entités principales sont les tâches à exécuter, les nœuds de calcul volontaires et la fonction d'assignation régissant leurs interactions. Ces entités sont définies par des caractéristiques capturant les contraintes spécifiques du VC. Nous les définissons dans les lignes qui suivent.

2.1.1.1 Tâche

L'ensemble des tâches est noté $\mathcal{T} = \{task_1, task_2, \dots, task_m\}$, où chaque tâche $task_i$ est un tuple $(Exe, D, QoS_{req}, Submit, Type, P, Son, F_{in}, F_{out})$ où :

- $Exe(task_i)$: est le temps d'exécution estimé, reflétant la charge computationnelle ;
- $D(task_i)$: est la deadline absolue, indiquant le temps maximal acceptable pour le résultat ;
- $QoS_{req}(task_i)$: désigne les exigences de qualité de service, un vecteur multi-dimensionnel (CPU, mémoire, espace disque et bande passante) ;
- $Submit(task_i)$: est l'instant de soumission, pour calculer l'urgence ;
- $Type(task_i)$: est la catégorie de la tâche, il est issu du type de calcul soumis. Ce paramètre influence les préférences des volontaires ;
- $P(task_i)$: désigne l'ensemble des tâches parentes (précédentes) dans le DAG de la tâche $task_i$, définissant les dépendances, une tâche sans parent est exécutable immédiatement alors que les tâches qui en possèdent pas doivent attendre leurs résultats de leur parents pour commencer leur exécution ;
- $Son(task_i)$: est l'ensemble des tâches enfants (suivantes) dans le DAG de la tâche $task_i$;
- $F_{in}(task_i)$: est l'ensemble des fichiers d'entrée, défini par leur nombre et leur taille totale ;
- $F_{out}(task_i)$: ensemble des fichiers de sortie attendus, également en nombre et taille.

Ces attributs modélisent la diversité des tâches, assurant une adéquation fine avec les nœuds hétérogènes.

2.1.1.2 Nœuds

L'ensemble des nœuds de calcul est noté $\mathcal{N} = \{N_1, N_2, \dots, N_n\}$, où chaque nœud N_j est un tuple $(Cap, A, C, H, Conf, AvgExe, Q, HistConn)$ défini par :

- $Cap(N_j)$: capacité de calcul, vecteur aligné avec QoS_{req} ;
- $A(N_j, t)$: disponibilité à l'instant t (1 si active, 0 sinon) ;
- $C(N_j, t)$: connectivité à l'instant t (1 si connecté, 0 sinon) ;
- $H(N_j)$: temps d'exécution cumulé historique ;
- $Conf(N_j)$: taux de confiance, pourcentage de résultats valides ;
- $AvgExe(N_j)$: temps d'exécution moyen par tâche ;

- Q_j : file d'attente locale, capacité maximale Γ_j ;
- $HistConn(N_j)$: historique des connexions/déconnexions.

Ces caractéristiques permettent d'évaluer la pertinence d'un nœud volontaire. Elles incluent l'historique des connexions/déconnexions ($HistConn(N_j)$), le taux de résultats valides ($Conf(N_j)$), le temps moyen d'exécution par tâche ($AvgExe(N_j)$) pour mesurer la performance computationnelle, et le temps d'exécution cumulé ($H(N_j)$). Ce dernier reflète l'ancienneté du volontaire et la durée d'activité de l'application de calcul sur sa machine, valorisant ainsi ses contributions.

2.1.1.3 Fonction d'Assignment

La fonction d'assignation formalise l'attribution d'une tâche à un volontaire dans un cadre mathématique rigoureux. Définie de l'ensemble des tâches $\mathcal{T}$ vers l'ensemble des volontaires $\mathcal{N}$, elle est donnée par la fonction $g(t, task_i)$:

$$g(t, task_i) = \begin{cases} N_j, & \text{si la tâche est assignée au volontaire } N_j, \\ \text{NULL}, & \text{si la tâche est rejetée.} \end{cases}$$

Elle permet de trouver à un instant t le volontaire N_j qui correspond le mieux à la tâche $task_i$. L'objectif des sections suivantes de ce chapitre est d'optimiser la valeur de $g(t, task_i) = N_j$ face aux contraintes et défis inhérents au système.

2.1.2 Modélisation de la Volatilité et Disponibilité

Pour répondre aux défis de la volatilité des nœuds, nous proposons une modélisation quantitative inspirée des approches réussies dans [15]. La volatilité et la disponibilité sont quantifiées via des métriques spécifiques qui permettent une évaluation dynamique des nœuds. Ces métriques, absentes dans les méthodes traditionnelles, garantissent une allocation adaptative par leur utilité dans la prise en compte de la probabilité que le volontaire soit disponible au moment de la terminaison ou qu'il soit connecté au système pour la réception ou le retour des résultats de la tâche avant sa deadline.

Métrique de volatilité : la métrique de volatilité $Vol(N_j, t)$ mesure la fréquence des changements d'état de connectivité d'un nœud N_j sur une fenêtre d'observation T_{obs} (par exemple, les dernières 24 heures) en fonction de l'instant présent t :

$$Vol(N_j, t) = \frac{\sum_{k=t-T_{obs}}^{t-1} |C(N_j, k) - C(N_j, k-1)|}{\max(1, \sum_{k=t-T_{obs}}^{t-1} C(N_j, k))},$$

où $C(N_j, k)$ est l'état de connectivité au temps k . Le numérateur compte les transitions entre états connecté et déconnecté, tandis que le dénominateur normalise par le temps total de connexion, évitant ainsi de pénaliser les nœuds rarement connectés mais stables. Cette métrique, bornée entre 0 et environ 2, pénalise davantage les nœuds instables tout en restant robuste face à des profils de connexion variés.

Métrique de disponibilité : la disponibilité, mesurant la proportion du temps pendant lequel le nœud est opérationnel pour le calcul qu'il soit connecté au système ou non. Elle est définie par :

$$Disp(N_j, t) = \frac{\sum_{k=t-T_{obs}}^{t-1} A(N_j, k)}{\max(1, t - T_{obs})},$$

où $A(N_j, k)$ indique si l'application de calcul est active. Cette métrique, bornée entre 0 et 1, évalue la disponibilité effective en tenant compte des périodes de connexion, évitant de pénaliser les nœuds pour leur indisponibilité lorsqu'ils sont déconnectés. Cette métrique permet de prioriser les nœuds opérationnels, contrairement aux méthodes comme MET qui se concentrent uniquement sur les performances passées [4].

Ces métriques, intégrées à la solution, permettent de quantifier précisément l'instabilité des nœuds, offrant une base pour des décisions d'ordonnancement robustes. La sous-section suivante complète cette modélisation en définissant la charge des files d'attente, un facteur clé pour équilibrer l'allocation des tâches.

Charge des Files d'Attente La gestion des files d'attente locales des nœuds est cruciale pour optimiser l'ordonnancement dans un environnement VC, où la surcharge ou la sous-utilisation du tampon local des nœuds peut entraîner des retards ou une inefficacité. Inspirée par [15], nous modélisons la charge de la file d'attente Q_j du nœud N_j au temps t via un ratio normalisé :

$$LoadRatio(Q_j, t) = \frac{|Q_j(t)|}{\Gamma_j},$$

où $|Q_j(t)|$ est le nombre de tâches dans la file et Γ_j est la capacité maximale de la file. Ce ratio, compris entre 0 et 1, reflète l'encombrement de la file, permettant d'éviter l'assignation de tâches à des nœuds déjà saturés.

Pour faciliter son intégration, nous discrétisons ce ratio en six niveaux distincts, adaptés à la complexité des décisions dans un environnement hétérogène :

$$L(Q_j, t) = \begin{cases} \text{Vide (V)}, & \text{si } LoadRatio(Q_j, t) = 0, \\ \text{Faible (F)}, & \text{si } 0 < LoadRatio(Q_j, t) \leq 0.25, \\ \text{Modérée (M)}, & \text{si } 0.25 < LoadRatio(Q_j, t) \leq 0.5, \\ \text{Élevée (E)}, & \text{si } 0.5 < LoadRatio(Q_j, t) \leq 0.75, \\ \text{Critique (C)}, & \text{si } 0.75 < LoadRatio(Q_j, t) < 1, \\ \text{Pleine (P)}, & \text{si } LoadRatio(Q_j, t) = 1. \end{cases}$$

Cette discrétisation, inspirée des niveaux de charge dans [15], simplifie l'espace d'états pour l'algorithme RL, tout en capturant les variations critiques de la charge. Elle permet de prioriser les nœuds avec des files peu chargées, réduisant les temps d'attente et améliorant l'équilibre de charge par rapport aux approches comme l'ordonnancement basé sur la localité, qui risque de créer des congestions [2].

Ces métriques de volatilité, disponibilité et charge des files d'attente forment un cadre cohérent pour modéliser les dynamiques complexes du VC. Elles permettent de surmonter les limites des méthodes traditionnelles, qui échouent à exploiter pleinement l'hétérogénéité et à gérer les fluctuations des nœuds [2]. La section suivante formalise ce problème comme un processus de décision markovien, en intégrant ces métriques dans un cadre d'optimisation pour maximiser la satisfaction globale à long terme.

2.1.3 Scores de Satisfaction pour l'Évaluation des Assignations

Pour évaluer la qualité d'une assignation de tâche et affiner progressivement les décisions d'ordonnancement au fil du temps via l'apprentissage par renforcement, nous définissons une série de scores de satisfaction spécifiques. Ces scores, inspirés des contraintes QoS et SSspecs dans [15], quantifient divers aspects de la compatibilité entre une tâche et un nœud, en tenant compte de la volatilité, de l'hétérogénéité et des contraintes temporelles.

Ils permettent une évaluation multi-critères qui surpasse les approches traditionnelles comme MET ou WRR, qui reposent sur des métriques simplifiées et statiques [1]. Chaque score est conçu pour produire une valeur normalisée, facilitant leur combinaison dans un score global optimisé par le RL.

2.1.3.1 Score de Satisfaction QoS

Le score de satisfaction QoS $QoSS(task_i, N_j)$ évalue la capacité d'un nœud N_j à répondre aux exigences de qualité de service d'une tâche $task_i$:

$$QoSS(task_i, N_j) = \frac{1}{1 + e^{-\alpha \cdot (QoS_{ratio} - 1)}},$$

où :

- $QoS_{ratio} = \min_{k \in \text{dimensions}} \frac{Cap(N_j)_k}{QoS_{req}(task_i)_k}$ est le ratio minimal des capacités du nœud par rapport aux exigences de la tâche sur toutes les dimensions .
- α est un paramètre de sensibilité contrôlant la steepness de la transition (typiquement $\alpha > 0$).

Cette fonction sigmoïde est choisie pour son comportement progressif : elle produit un score entre 0 et 1, avec une valeur de 0.5 lorsque $QoS_{ratio} = 1$ (capacités exactement égales aux exigences), augmentant vers 1 pour des capacités supérieures et diminuant vers 0 pour des insuffisances. Sa dérivabilité facilite l'apprentissage par gradients dans le RL, et sa forme asymétrique autour de 1 pénalise plus sévèrement les manques que les excès, alignée avec les besoins pratiques du VC où une sous-capacité peut causer des échecs [4].

2.1.3.2 Score de Deadline

Le score de deadline $DeadlineS(task_i, N_j, t)$ évalue la probabilité que le volontaire respecte la deadline de la tâche, en tenant compte du temps d'attente et d'exécution ajusté pour la volatilité :

$$DeadlineS(task_i, N_j, t) = \begin{cases} \tanh \left(\gamma \cdot \frac{D(task_i) - t - WT(N_j, t) - Exe(task_i) \cdot (1 + \beta \cdot Vol(N_j, t))}{Exe(task_i)} \right), & \text{si } > 0, \\ -1, & \text{sinon,} \end{cases}$$

où :

- $D(task_i)$ est la deadline absolue de la tâche.
- $WT(N_j, t) = \sum_{task_k \in Q_j(t)} Exe(task_k)$ est le temps d'attente estimé dans la file Q_j .
- $Exe(task_i)$ est le temps d'exécution estimé de la tâche.
- $Vol(N_j, t)$ est le score de volatilité du nœud.
- γ et β sont des paramètres d'ajustement ($\gamma > 0$ pour la sensibilité, $\beta > 0$ pour pénaliser la volatilité).

La fonction tangente hyperbolique ($\tanh$) est sélectionnée pour son comportement borné entre -1 et 1, avec une sensibilité accrue près de zéro, distinguant finement les situations marginales (légèrement en avance ou en retard). Elle intègre une pénalité proportionnelle à la volatilité via β , augmentant le temps d'exécution effectif pour les nœuds instables, ce qui reflète les interruptions potentielles. Ce choix symétrique pénalise les retards de manière proportionnelle mais limitée, favorisant une optimisation équilibrée par rapport aux méthodes comme le tourniquet, qui ignorent les deadlines dynamiques [2].

2.1.3.3 Score de Fiabilité Historique

Le score de fiabilité historique $ReliabilityS(N_j)$ combine le taux de confiance et l'expérience accumulée du nœud :

$$ReliabilityS(N_j) = Conf(N_j) \cdot \frac{2}{1 + e^{-\delta \cdot H(N_j)}},$$

où :

- $Conf(N_j)$ est le taux de confiance (pourcentage de résultats valides).
- $H(N_j)$ est le temps total d'exécution cumulé.
- $\delta > 0$ contrôle la vitesse de convergence vers la valeur maximale.

Cette formulation multiplicative assure que les nœuds peu fiables ($Conf(N_j) < 1$) obtiennent des scores bas, tandis que la sigmoïde adaptée tend vers 2 pour les nœuds expérimentés, normalisant le score final entre 0 et 1 (puisque $Conf(N_j) \leq 1$). Le choix de la sigmoïde permet une transition progressive : scores proches de 0 pour les nouveaux nœuds ($H(N_j) \approx 0$) et augmentation vers la limite pour les éprouvés, distinguant ainsi l'expérience de la fiabilité brute. Cela surmonte les limites des méthodes probabilistes, qui négligent souvent l'historique cumulatif [9].

2.1.3.4 Score de Réception à Temps

Le score de réception à temps $ReceptionS(task_i, N_j, t)$ évalue si le nœud recevra la tâche suffisamment tôt :

$$ReceptionS(task_i, N_j, t) = \begin{cases} 1, & \text{si } C(N_j, t) = 1, \\ P(t' - t \leq 0.5 \cdot Exe(task_i) | HistConn(N_j)), & \text{sinon,} \end{cases}$$

où :

- t' est le prochain temps de connexion prévu.
- $P(\cdot | HistConn(N_j))$ est la probabilité empirique estimée à partir de l'historique des connexions :

$$P(t' - t \leq \tau | HistConn(N_j)) = \frac{\text{Nombre de périodes de déconnexion} \leq \tau \text{ dans } HistConn(N_j)}{\text{Nombre total de périodes de déconnexion dans } HistConn(N_j)},$$

avec $\tau = 0.5 \cdot (D(task_i) - t)$.

Cette approche probabiliste empirique est choisie pour sa base statistique réelle, son but est d'encourager les volontaires qui sont connectés pendant l'attribution ou très proche de t et pénalise ceux qui se connecteront à plus de 50% du temps écoulé avant la deadline. Elle produit un score entre 0 et 1, avec un seuil critique de 50% de la durée entre l'instant d'attribution et la deadline pour accepter les déconnexions courtes, pénalisant les nœuds volatils sans les exclure systématiquement, favorisant ainsi une allocation flexible.

2.1.3.5 Score de Retour à Temps

Le score de retour à temps $ReturnS(task_i, N_j, t)$ évalue si le nœud retournera les résultats à temps malgré sa volatilité :

$$ReturnS(task_i, N_j, t) = \begin{cases} 1, & \text{si } Vol(N_j, t) \leq \theta_{vol}, \\ 1 - Vol(N_j, t) \cdot \frac{\min(Exe(task_i), \bar{E})}{5 \cdot \bar{E}}, & \text{sinon,} \end{cases}$$

où :

- θ_{vol} est un seuil de volatilité acceptable.
- $\bar{E}$ est le temps d'exécution moyen des tâches.

Cette formulation linéaire avec seuil est sélectionnée pour sa simplicité et son comportement progressif : elle accorde un score maximal aux nœuds stables et pénalise proportionnellement à la volatilité et au temps d'exécution normalisé (plafonné pour éviter sur-pénalisation des tâches longues). Le facteur 5 calibre la sensibilité, assurant une décroissance modérée, contrairement aux méthodes rigides comme le seuil-based qui excluent sans nuance [2].

2.1.3.6 Score de Satisfaction Globale

Le score de satisfaction globale $S(task_i, g(t, task_i))$ combine ces métriques pour évaluer une assignation :

$$S(task_i, g(t, task_i)) = \begin{cases} -b, & \text{si } g(t, task_i) = \text{NULL}, \\ w_1 \cdot QoSS(task_i, N_j) + w_2 \cdot DeadlineS(task_i, N_j, t) + \\ w_3 \cdot ReliabilityS(N_j) + w_4 \cdot ReceptionS(task_i, N_j, t) + \\ w_5 \cdot ReturnS(task_i, N_j, t) + w_6 \cdot Disp(N_j, t), & \text{si } g(t, task_i) = N_j \text{ et } A(N_j, t) = 1, \\ 0, & \text{si } g(t, task_i) = N_j \text{ et } A(N_j, t) = 0, \end{cases}$$

où $w_1, \dots, w_6$ sont des poids ajustables avec $\sum w_i = 1$, et $b > 0$ est une pénalité pour les rejets. Ce score pondéré intègre tous les aspects critiques, pénalisant les rejets pour encourager les assignations raisonnables et attribuant zéro aux nœuds indisponibles, favorisant une optimisation équilibrée.

2.1.3.7 Problème d'Optimisation

L'objectif est de maximiser la satisfaction moyenne à long terme :

$$\text{Maximiser : } \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \sum_{i \in \mathcal{T}} S(task_i, g(t, task_i)),$$

sous les contraintes :

$$|Q_j(t)| + 1_{g(t, task_i)=N_j} \leq \Gamma_j \quad \forall j \in [n], \quad (2.1)$$

$$ReceptionS(task_i, N_j, t) \geq \theta_{reception} \quad \text{si } g(t, task_i) = N_j, \quad (2.2)$$

$$ReturnS(task_i, N_j, t) \geq \theta_{return} \quad \text{si } g(t, task_i) = N_j, \quad (2.3)$$

où $\theta_{reception}$ et θ_{return} sont des seuils minimaux, et 1 est l'indicatrice.

Ce problème, similaire à des variantes d'ordonnancement avec contraintes (e.g., job shop scheduling avec deadlines et ressources hétérogènes), est NP-complet et ne peut être résolu en temps polynomial pour des instances générales, justifiant l'utilisation d'une approche heuristique adaptative comme le RL [13]. Cette formalisation pave la voie à la modélisation MDP dans la section suivante, où ces éléments sont intégrés pour une résolution dynamique.

2.2 Solution basée sur Apprentissage par Renforcement

Après avoir modélisé notre système et formalisé notre problème avec les contraintes posées par l'environnement, la prochaine étape est la solution au problème. Cela passe par le RL où nous commençons par formaliser le problème en tant que MDP, puis détaillons l'architecture A3C adaptée, en mettant en évidence ses composants et ses mécanismes d'adaptation.

2.2.1 Formalisation sous un Processus de Décision Markovien (MDP)

La modélisation du problème d'ordonnancement comme un MDP permet de capturer les dynamiques complexes du VC, caractérisées par l'incertitude des connexions et la variabilité des capacités des nœuds. Contrairement aux méthodes traditionnelles comme FCFS ou MET, qui reposent sur des hypothèses statiques [6], le cadre MDP offre une structure formelle pour optimiser les prises de décisions séquentielles en temps réel, en intégrant les métriques de volatilité, disponibilité et charge définies précédemment. Le MDP est défini par le tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ [11], adapté comme suit :

- **État** ($\mathcal{S}$) : L'état du système au temps t , noté $s(t)$, inclut :

$$s(t) = [task_i, \{L(Q_j, t)\}_{j \in [n]}, \{C(N_j, t)\}_{j \in [n]}, \{A(N_j, t)\}_{j \in [n]}],$$

où $task_i$ contient les caractéristiques de la tâche courante ($Exe(task_i)$, $D(task_i)$, $QoS_{req}(task_i)$, $Submit(task_i)$, $type(task_i)$), $\{L(Q_j, t)\}$ sont les charges discrétisées des files d'attente, $\{C(N_j, t)\}$ les états de connectivité, et $\{A(N_j, t)\}$ les disponibilités des nœuds. Cette représentation capture l'hétérogénéité et la volatilité en temps réel.

- **Actions** ($\mathcal{A}$) : L'espace des actions au temps t est :

$$a(t) \in \{N_1, N_2, \dots, N_n, \text{NULL}\},$$

correspondant à l'assignation de $task_i$ à un nœud N_j ou à son rejet (NULL).

- **Transitions** ($T(s'|s, a)$) : Les probabilités de transition modélisent la dynamique du système, incluant les changements de connectivité ($C(N_j, t)$) et de disponibilité ($A(N_j, t)$), estimées à partir des historiques $HistConn(N_j)$. Ces transitions reflètent l'incertitude des nœuds volatils.
- **Récompense** (R) : La récompense immédiate est donnée par le score de satisfaction globale défini précédemment :

$$R(t) = S(task_i, g(t, task_i)),$$

intégrant $QoSS$, $DeadlineS$, $ReliabilityS$, $ReceptionS$, $ReturnS$, et $Disp$, avec une pénalité pour les rejets ($-b$) et un score nul pour les nœuds indisponibles.

- **Facteur d'actualisation** (γ) : $\gamma \in [0, 1]$ pondère les récompenses futures, favorisant une optimisation à long terme (typiquement $\gamma \approx 0.9$) [12].

Cette formalisation MDP, permet d'optimiser la satisfaction globale tout en gérant l'incertitude inhérente au VC. En intégrant des métriques spécifiques (volatilité, disponibilité, charge), elle pose les bases d'une solution RL capable d'apprendre des politiques adaptatives. La sous-section suivante détaille l'architecture A3C adaptée pour opérationnaliser ce cadre.

2.2.2 Architecture A3C Adaptée

L'algorithme Asynchronous Advantage Actor-Critic (A3C) est choisi pour sa capacité à gérer des environnements complexes et dynamiques, comme démontré dans le VEC, contrairement aux méthodes probabilistes ou évolutionnaires, qui souffrent de surcharges computationnelles [9], A3C combine apprentissage asynchrone et stabilité pour optimiser l'ordonnancement dans le VC et il est facilement déployable sur des machines aux capacités limitées. Notre architecture est adaptée pour intégrer les métriques de volatilité et d'hétérogénéité, avec des mécanismes spécifiques pour une allocation robuste et efficace.

2.2.2.1 Composants Principaux

L'architecture A3C repose sur deux réseaux neuronaux principaux :

- **Réseau Acteur** : Détermine la politique d'assignation $\pi(a|s)$, qui associe un état $s(t)$ à une probabilité d'action. Ce réseau apprend à sélectionner des actions maximisant la récompense à long terme, en intégrant les scores de satisfaction.
- **Réseau Critique** : Évalue la valeur d'état $V(s)$, estimant la récompense cumulée attendue à partir de l'état $s(t)$. Il guide l'acteur en calculant l'avantage des actions, améliorant la stabilité de l'apprentissage.

Ces composants, entraînés de manière asynchrone sur plusieurs threads, permettent une exploration rapide et une convergence efficace, surpassant les approches centralisées comme DQN qui souffrent de biais de surestimation [13].

2.2.2.2 Mécanismes de Gestion de la Volatilité

Pour contrer la volatilité des nœuds, nous intégrons deux mécanismes spécifiques dans la politique A3C, assurant une robustesse face aux déconnexions fréquentes, un défi non résolu par les méthodes traditionnelles comme la réplication systématique [8].

Diversification Adaptative : la politique d'assignation est ajustée dynamiquement pour équilibrer performance et fiabilité :

$$\pi(N_j|s(t)) \propto \exp(\kappa \cdot S'(task_i, N_j)),$$

où :

$$S'(task_i, N_j) = S(task_i, N_j) - \rho \cdot Vol(N_j, t) \cdot Urgence(task_i),$$

et :

$$Urgence(task_i) = \frac{1}{\max(D(task_i) - t - Exe(task_i), \epsilon)},$$

avec $\kappa > 0$ contrôlant l'exploration, $\rho > 0$ l'impact de la volatilité, et $\epsilon > 0$ un petit terme pour éviter les divisions par zéro. Ce mécanisme favorise les nœuds stables pour les tâches urgentes (forte $Urgence(task_i)$) tout en explorant les nœuds volatils pour les tâches moins critiques, réduisant les interruptions par rapport aux approches comme le tourniquet [2].

Réplication Sélective : pour les tâches critiques, définies par une deadline serrée et une priorité élevée, nous appliquons une réplication sélective :

Si $(D(task_i) - t) < (1 + \text{marge}) \cdot Exe(task_i)$ et $\text{priorité}(task_i) > \text{seuil}$, alors assigner à deux nœuds.

Cette stratégie, inspirée de [15], améliore la robustesse sans gaspiller les ressources comme dans la réplication systématique de BOINC [8], en ciblant uniquement les tâches à risque élevé.

2.2.2.3 Mécanismes d'Adaptation à l'Hétérogénéité

Pour exploiter pleinement l'hétérogénéité des nœuds, nous intégrons deux mécanismes dans A3C, permettant une allocation optimisée des tâches en fonction des capacités et performances historiques.



Spécialisation des Nœuds : nous ajustons le score QoS pour refléter les performances historiques par type de tâche :

$$QoS_{specialized}(task_i, N_j) = QoS(task_i, N_j) \cdot (1 + \eta \cdot HistoPerf(type(task_i), N_j)),$$

où :

$$HistoPerf(type, N_j) = \frac{\overline{Exe_{attendu}(type)}}{\overline{Exe_{rel}(type, N_j)}},$$

et $\eta > 0$ contrôle l'impact de l'historique. Cette métrique favorise les nœuds ayant démontré une efficacité pour des types spécifiques de tâches (e.g., calcul intensif), surmontant l'incapacité des méthodes comme l'assignation aléatoire à exploiter l'hétérogénéité [2].

Équilibrage Charge-Fiabilité : nous ajustons la perception de la charge en fonction de la fiabilité :

$$LoadFactor(N_j) = \frac{|Q_j|}{\Gamma_j} \cdot (2 - ReliabilityS(N_j)),$$

orientant les tâches vers les nœuds fiables, même légèrement plus chargés, pour éviter les échecs fréquents observés dans les approches comme FCFS [6]. Ce mécanisme garantit une répartition équilibrée tout en maximisant la fiabilité.

Ces mécanismes, intégrés dans l'architecture A3C, permettent une allocation adaptative et robuste, garantissant une amélioration des performances et de la résilience par rapport aux méthodes de la littérature. La section suivante présentera l'algorithme d'apprentissage et une synthèse des avantages de cette approche.

2.3 Solution Algorithmique et Perspectives

Après avoir donné la solution au problème, nous donnons maintenant les étapes à suivre pour sa mise en œuvre, cela passe par un algorithme. En s'appuyant sur la formalisation MDP et l'architecture A3C décrites dans la section 2.2, nous proposons un algorithme qui opérationnalise ces mécanismes pour optimiser l'ordonnancement. Une synthèse des avantages de l'approche, comparée aux méthodes traditionnelles, est ensuite présentée.

2.3.1 Algorithme d'Apprentissage

L'algorithme A3C adapté exploite l'apprentissage asynchrone pour apprendre une politique d'ordonnancement optimale, en intégrant les scores de satisfaction et les mécanismes de diversification adaptative, réplique sélective, spécialisation des nœuds, et équilibrage charge-fiabilité. Le pseudocode suivant formalise l'algorithme, en détaillant les étapes clés de l'apprentissage et de la gestion des interruptions dues à la volatilité.

Cet algorithme intègre les mécanismes de diversification adaptative (via S') et de réplique sélective pour gérer la volatilité, ainsi que la spécialisation des nœuds et l'équilibrage charge-fiabilité pour exploiter l'hétérogénéité. La mise à jour asynchrone des paramètres globaux réduit la surcharge computationnelle, contrairement aux approches centralisées comme DQN [13], et permet une adaptabilité en temps réel face aux fluctuations des nœuds, comblant ainsi les limites des méthodes statiques comme WRR [1, 4]. L'utilisation des historiques et des scores dynamiques garantit une allocation optimisée, même dans des environnements fortement volatils.





Algorithmme 1 A3C Adapté pour l'Ordonnancement avec Gestion de la Volatilité et Hétérogénéité

```

1: Initialiser les paramètres  $\theta$  du réseau Acteur et  $\theta_v$  du réseau Critique.
2: Initialiser les paramètres globaux partagés  $\theta_{\text{global}}$  et  $\theta_{v,\text{global}}$ .
3: pour chaque épisode faire
4:   Synchroniser les paramètres locaux :  $\theta \leftarrow \theta_{\text{global}}, \theta_v \leftarrow \theta_{v,\text{global}}$ .
5:   Réinitialiser les gradients :  $d\theta \leftarrow 0, d\theta_v \leftarrow 0$ .
6:   pour  $t = 1$  à  $T$  (horizon de l'épisode) faire
7:     Observer l'état  $s(t) = [task_i, \{L(Q_j, t)\}_{j \in [n]}, \{C(N_j, t)\}_{j \in [n]}, \{A(N_j, t)\}_{j \in [n]}]$ .
8:     Calculer les scores de satisfaction  $S'(task_i, N_j)$  pour chaque nœud disponible,
      ajustés pour la volatilité et l'urgence.
9:     Sélectionner l'action  $a(t) \in \{N_1, \dots, N_n, \text{NULL}\}$  selon la politique  $\pi(a|s; \theta)$ .
10:    si  $task_i$  est critique ( $(D(task_i) - t) < (1 + \text{marge}) \cdot Exe(task_i)$ ) et priorité élevée)
      alors
11:      Sélectionner un second nœud pour la réplication sélective.
12:    fin si
13:    Exécuter l'action  $a(t)$  et observer la récompense  $R(t) = S(task_i, g(t, task_i))$ .
14:    si un nœud se déconnecte pendant l'exécution alors
15:      si reconnexion avant  $0.5 \cdot Exe(task_i)$  alors
16:        Continuer l'exécution.
17:      sinon
18:        Réassigner la tâche selon la politique de récupération (e.g., sélection
        d'un nœud alternatif).
19:        Réajuster la récompense pour refléter l'interruption ( $R(t) \leftarrow R(t) -$ 
        pénalité).
20:      fin si
21:    fin si
22:    Mettre à jour les historiques ( $HistConn(N_j), H(N_j), Conf(N_j)$ ) et recalculer
    les métriques ( $Vol(N_j, t), Disp(N_j, t)$ ).
23:    Calculer l'avantage :  $A(s, a) = R(t) + \gamma V(s'; \theta_v) - V(s; \theta_v)$ .
24:    Mettre à jour les gradients :  $d\theta \leftarrow d\theta + \nabla_{\theta} \log \pi(a|s; \theta) \cdot A(s, a), d\theta_v \leftarrow d\theta_v +$ 
     $\nabla_{\theta_v} (R(t) + \gamma V(s'; \theta_v) - V(s; \theta_v))^2$ .
25:    fin pour
26:    Mettre à jour les paramètres globaux :  $\theta_{\text{global}} \leftarrow \theta_{\text{global}} + \alpha \cdot d\theta, \theta_{v,\text{global}} \leftarrow \theta_{v,\text{global}} +$ 
     $\alpha \cdot d\theta_v$ .
27: fin pour

```





2.3.2 Synthèse et Avantages de l'Approche

Notre approche, basée sur une modélisation MDP et une architecture A3C adaptée, répond aux limites des méthodes traditionnelles en offrant une solution robuste et adaptative pour l'ordonnancement dans les systèmes VC. Les principaux avantages sont les suivants :

- **Gestion de la volatilité** : les mécanismes de diversification adaptative et de réplication sélective réduisent l'impact des déconnexions fréquentes grâce à l'attribution intelligente qu'ils engendrent. Ils limitent le gaspillage de ressources tout en optimisant l'utilisation de celles qui sont déjà allouées.
- **Exploitation de l'hétérogénéité** : la spécialisation des nœuds et l'équilibrage charge-fiabilité permettent une allocation intelligente, assignant les tâches aux nœuds les plus adaptés en fonction de leur historique et de leurs capacités, contrairement aux méthodes naïves comme FCFS [6].
- **Adaptabilité en temps réel** : l'apprentissage asynchrone d'A3C, combiné à des métriques dynamiques, permet des ajustements continus face aux fluctuations.
- **Robustesse et scalabilité** : la structure décentralisée de l'apprentissage A3C évite les goulots d'étranglement lors de l'apprentissage. De plus l'utilisation des réseaux de neurones permet d'obtenir un ordonnanceur qui gère aisément les systèmes à grande échelles.

Ces avantages positionnent notre approche comme une solution idéale pour transformer les ressources dispersées et instables du VC en une puissance de calcul cohérente et performante. Les résultats prometteurs d'A3C dans le VEC [15], avec 96.55% de satisfaction QoS à faible taux d'arrivée, suggèrent que notre adaptation peut atteindre des performances similaires, voire supérieures, dans le contexte VC.

Conclusion

Ce chapitre a présenté une approche innovante pour l'ordonnancement des tâches dans les systèmes de calcul sur machines volontaires, en s'appuyant sur une modélisation MDP rigoureuse et une architecture A3C adaptée. Cette approche garantit une terminaison rapide des tâches et une satisfaction globale élevée, même dans des environnements fortement instables. Le chapitre suivant évaluera expérimentalement ces performances, en testant notre algorithme sur des workflows réels dans un système de calcul réel afin de confirmer sa robustesse et son efficacité par rapport aux baselines existantes.

Expérimentation et résultats

Sommaire

Introduction	28
3.1 Méthodologie expérimentale	28
3.1.1 Conception de l'expérimentation	29
3.1.2 Protocole expérimental	30
3.2 Mise en œuvre de l'approche proposée	31
3.2.1 Système utilisé	31
3.2.2 Workflows	31
3.2.3 Caractéristiques des volontaires	32
3.3 Résultats obtenus et interprétation	32
3.3.1 Présentation des résultats	32
3.3.2 Comparaison des résultats	33
3.3.3 Interprétation et discussion	34
Conclusion	35

Introduction

Dans le chapitre précédent 2, nous avons proposé une solution d'ordonnancement des tâches au sein de workflows en VC, reposant sur l'apprentissage par renforcement et utilisant l'algorithme A3C. L'objectif de ce chapitre est de valider les performances de cette solution à travers une expérimentation pratique, en comparant ses résultats aux méthodes traditionnelles issues de la littérature. Pour ce faire, nous présenterons dans ce qui suit : la méthodologie expérimentale, la mise en œuvre pratique de notre solution, les résultats obtenus, leur analyse comparative avec les méthodes de référence, ainsi qu'une discussion approfondie des implications et une conclusion sur les perspectives futures.

3.1 Méthodologie expérimentale

Cette section décrit le cadre méthodologique de l'expérimentation visant à évaluer les performances de notre approche. Elle détaille la conception du système expérimental, incluant les participants et l'environnement, ainsi que le protocole expérimental utilisé pour tester notre solution.



3.1.1 Conception de l'expérimentation

Pour tester notre approche, nous avons utilisé un environnement de calcul sur machines volontaires réel. Ses composants et ses participants sont décrits dans ce qui suit.

3.1.1.1 Participants

L'expérimentation repose sur la participation de volontaires, recrutés parmi des étudiants via des communiqués officiels et des mini-conférences organisées dans des amphithéâtres et les salles de cours. Ces machines présentent une diversité notable, tant au niveau des systèmes d'exploitation que des capacités matérielles, cette hétérogénéité reflète bien les conditions réelles du VC. Dans la suite nous présentons l'environnement logiciel dans lequel notre expérimentation s'est faite.

3.1.1.2 Environnement

Le système expérimental est conçu pour respecter les principes fondamentaux du VC, tout en restant suffisamment générique pour être adaptable à différentes implémentations. La conception du système repose sur une architecture hybride dont les composants sont les suivants :

- **Serveur central** : un serveur unique agit comme le répertoire central d'informations du système. Il est responsable de la coordination et de la transmission de tous les messages échangés entre les entités du système, garantissant une communication fluide et centralisée.
- **Volontaires** : les volontaires sont des applications installées sur les machines des participants. Une fois connectées au réseau, ces applications s'inscrivent automatiquement auprès du serveur central et restent en attente de tâches à exécuter. Elles représentent les nœuds de calcul du système VC.
- **Clients** : les clients sont des entités soumettant des workflows à exécuter. C'est au niveau des clients que l'agent A3C est déployé pour gérer l'ordonnancement des tâches, en s'appuyant sur les informations fournies par le serveur.

Le fonctionnement conceptuel du système est le suivant : au démarrage, le serveur central est initialisé et devient opérationnel. Les applications volontaires, dès qu'elles se connectent au réseau, s'inscrivent auprès du serveur et signalent leur disponibilité pour recevoir des tâches. De leur côté, les clients s'inscrivent également auprès du serveur. Une fois leur inscription validée, ils soumettent leurs workflows, accompagnés d'estimations des ressources nécessaires pour chaque tâche. En réponse, le serveur fournit une liste des volontaires disponibles, incluant leur charge actuelle. L'agent A3C, intégré au client, prend alors en charge l'ordonnancement des tâches en assignant dynamiquement chaque tâche à un volontaire pendant l'épisode d'exécution du workflow. Une fois les tâches exécutées, les résultats sont renvoyés au client, qui se charge de les assembler pour produire le résultat final du workflow. Cette conception, volontairement générique, permet de tester notre approche dans tout système respectant ces principes, indépendamment des spécificités techniques des machines ou du réseau utilisé.

3.1.1.3 Workflows et tâches

L'expérimentation utilise des workflows sans dépendances, c'est-à-dire des ensembles de tâches indépendantes les unes des autres donc ne nécessitant pas de contraintes d'ordre d'exécution. Les tâches ont des exigences de ressources relativement faibles, adaptées aux capacités typiques des machines personnelles des volontaires. Cette simplicité dans la structure des workflows permet de se concentrer sur l'efficacité de l'ordonnancement





par A3C, tout en minimisant les complexités liées à la gestion des dépendances. Les estimations des ressources requises sont fournies par les clients lors de la soumission des workflows.

3.1.2 Protocole expérimental

Le protocole expérimental définit les étapes et les conditions dans lesquelles les tests sont réalisés pour évaluer les performances de l'agent A3C par rapport aux méthodes traditionnelles. Il vise à garantir la reproductibilité des résultats tout en testant l'approche dans des conditions représentatives du VC. Les détails du protocole, incluant les scénarios testés, les algorithmes comparés, et les métriques d'évaluation, sont décrits dans la suite de cette section.

3.1.2.1 Scénarios expérimentaux

L'expérimentation est structurée en trois scénarios de charge distincts : faible, moyenne et forte. Cette répartition permet d'évaluer la performance de l'ordonnancement dans des conditions variées, représentatives des fluctuations typiques des systèmes VC. Le scénario de faible charge simule un environnement avec un nombre limité de tâches et de volontaires, reflétant des périodes de faible activité où les ressources disponibles excèdent les besoins. Le scénario de charge moyenne représente un équilibre entre le nombre de tâches et les ressources disponibles, mettant à l'épreuve l'aptitude de l'algorithme à gérer une charge modérée dans des conditions courantes. Enfin, le scénario de forte charge introduit un grand nombre de tâches, dépassant potentiellement les capacités disponibles, afin de tester la robustesse et l'efficacité de l'ordonnancement dans des situations critiques où la volatilité et la concurrence pour les ressources sont élevées. Cette gradation des scénarios permet de capturer un spectre complet des dynamiques du VC, garantissant une évaluation exhaustive des performances de l'agent A3C face à des contextes variés.

3.1.2.2 Algorithmes comparés

Pour évaluer l'efficacité de notre approche, l'agent A3C est comparé à deux algorithmes de référence : *First-Come, First-Served* (FCFS) et *Mean Execution Time* (MET). Le choix de ces algorithmes est motivé par leur pertinence et leur prévalence dans la littérature sur l'ordonnancement en VC et systèmes distribués. FCFS, en tant qu'approche naïve, est largement utilisé dans les systèmes VC en raison de sa simplicité et de sa facilité de mise en œuvre, ce qui en fait une baseline incontournable pour évaluer toute nouvelle méthode. MET, quant à lui, représente une approche heuristique plus sophistiquée, qui priorise les tâches en fonction de leur temps d'exécution estimé, offrant ainsi une alternative plus intelligente que FCFS. Ce choix permet de confronter A3C à une méthode basée sur la connaissance a priori, souvent adoptée dans les systèmes où des estimations de performance sont disponibles. Cette sélection garantit une évaluation rigoureuse, ancrée dans des standards reconnus.

3.1.2.3 Métriques d'évaluation

Les performances des algorithmes sont évaluées à l'aide de plusieurs métriques, choisies pour refléter les objectifs d'optimisation du VC : le *makespan*, l'utilisation des ressources, la latence réseau, et le taux d'échec des tâches. Ces métriques sont collectées à l'aide d'outils de journalisation au niveau du serveur, et d'outils de suivi des performances installés sur les machines des volontaires, qui mesurent l'utilisation des ressources locales.





Des outils d'analyse de réseau sont également utilisés pour quantifier les transferts de données. Ces choix permettent une évaluation complète et objective des performances, capturant à la fois l'efficacité temporelle, l'exploitation des ressources, et la robustesse face aux défaillances potentielles dans un environnement VC.

3.2 Mise en œuvre de l'approche proposée

Cette section décrit la mise en œuvre pratique de l'expérimentation, en détaillant le système de calcul volontaire utilisé, les caractéristiques des workflows exécutés, et les spécifications des volontaires ayant participé. L'objectif est de fournir une vue d'ensemble de l'environnement technique mis en place pour tester l'efficacité de l'agent A3C dans l'ordonnancement des tâches, en s'appuyant sur une implémentation concrète adaptée aux contraintes du VC.

3.2.1 Système utilisé

L'expérimentation s'appuie sur le système de calcul volontaire *vcuy1*, développé à l'Université de Yaoundé I, entièrement implémenté en Python pour garantir une intégration fluide et une maintenance aisée. Ce système respecte la conception générique décrite dans la section 3.1.1.2, avec les composants suivants :

- **Serveur central** : l'application serveur, nommée *coordonateur*¹, utilise une architecture de communication basée sur le modèle *publish/subscribe* (pub/sub) implémentée via Redis, permettant une gestion efficace des messages échangés entre les entités du système. Une base de données MongoDB est intégrée pour stocker les informations relatives aux workflows, aux volontaires, et aux résultats des tâches.
- **Application client** : également appelée *manager*², l'application client est développée en Python. C'est au sein de cette application que l'agent A3C est déployé pour gérer l'ordonnancement des tâches. L'agent est implémenté à l'aide des bibliothèques PyTorch et Gym d'OpenAI pour l'entraînement, utilisant un réseau de neurones de type perceptron multicouche (MLP) avec une architecture composée de 512 unités dans la première couche cachée, 256 unités dans la seconde, une activation ReLU, une sortie *softmax* pour l'acteur, et une sortie scalaire pour le critique.
- **Application volontaire**³ : développée en Python, cette application est installée sur les machines des volontaires. Elle utilise la conteneurisation via Docker pour exécuter les tâches dans un environnement isolé du système hôte, garantissant ainsi la compatibilité et la sécurité des exécutions, quelles que soient les configurations matérielles et logicielles des machines.

3.2.2 Workflows

Les workflows exécutés dans l'expérimentation sont basés sur la simulation épidémiologique OpenMalaria [17], conçue pour modéliser des stratégies de lutte contre le paludisme. Ces workflows sont composés de tâches indépendantes, conformément à la description de la section 3.1.1.3, avec des exigences de ressources relativement faibles, adaptées aux capacités des machines personnelles des volontaires.

1. <https://github.com/VC-UY/coordinator-app-2025.git>

2. <https://github.com/VC-UY/manager-app-2025.git>

3. <https://github.com/VC-UY/volunteer-app-2025.git>



3.2.3 Caractéristiques des volontaires

Les volontaires ayant participé à l'expérimentation utilisent des machines personnelles allant des desktops aux laptops avec les systèmes d'exploitation Windows et Linux. Sur la base des données collectées, un total de 42 machines ont été utilisées. Les tableaux suivants résument leur répartition selon le nombre total de machines, les types de machines, les systèmes d'exploitation, la mémoire vive (RAM), et l'espace disque total.

TABLE 3.1 – Répartition par type de machine et RAM

Laptops	Desktops	Linux	Windows	4 Go	12 Go	15 Go	30 Go
39	3	32	10	17	3	6	2

Ces tableaux illustrent la diversité des ressources matérielles et logicielles des volontaires, avec une prédominance de laptops (88.09 %), de systèmes Linux (76.19 %), et une variation significative de la mémoire vive (de 4 à 30 Go) et de l'espace disque (de 27 648 à 471 040 Mo). Cette hétérogénéité souligne l'importance d'un ordonnancement adaptatif, comme celui proposé par l'agent A3C, pour optimiser l'utilisation des ressources dans un environnement VC.

3.3 Résultats obtenus et interprétation

Cette section présente les résultats obtenus lors de l'expérimentation réelle avec des cadets académiques de l'Université de Yaoundé I, pour trois scénarios de charge : faible (20 volontaires, 15 tâches), moyenne (20 volontaires, 60 tâches), et forte (42 volontaires, 160 tâches). Les performances des algorithmes FCFS (*First-Come, First-Served*), MET (*Mean Execution Time*), et A3C sont analysées en termes de *makespan* (minutes), utilisation des ressources (%), moyenne par volontaire, et quantité de données échangées (Mo, à minimiser). Le taux d'échec est évoqué dans les explications des graphiques, suivis d'une comparaison quantitative des algorithmes.

3.3.1 Présentation des résultats

Les résultats sont présentés par algorithme, pour chaque scénario de charge, avec des graphiques illustrant le *makespan*, l'utilisation des ressources, et les taux d'échec. Chaque tâche a un temps d'exécution variant de 5 à 25 minutes, avec des fichiers d'entrée/sortie d'environ 500 Ko.

FCFS : pour le scénario de faible charge FCFS atteint un *makespan* de 90 minutes, une utilisation des ressources de 35 %, et une quantité de données échangées de 7.3 Mo, sans échec. En charge moyenne, le *makespan* augmente à 163.7 minutes, l'utilisation à 65 %, et les données échangées à 29.3 Mo, avec un taux d'échec de 2 %. En forte charge, le *makespan* atteint 153 minutes, l'utilisation 73 %, et les données échangées 78.1 Mo, avec un taux d'échec de 5 %. Par exemple, un cadet exécute une tâche en 15 minutes en faible charge, mais FCFS surcharge les machines lentes en forte charge.

MET : pour la faible charge, MET atteint un *makespan* de 102.6 minutes, une utilisation de 42 %, et une quantité de données échangées de 7.3 Mo, sans échec. En charge moyenne, le *makespan* est de 150 minutes, l'utilisation de 82 %, et les données échangées de 29.3 Mo, avec un taux d'échec de 1.8 %. En forte charge, MET atteint un *makespan* de 138.8





minutes, une utilisation de 80.78 %, et 78.1 Mo de données échangées, avec un taux d'échec de 3.8 %. Par exemple, un cadet exécute 3 tâches en charge moyenne, MET optimisant le débit mais sous-utilisant certaines machines.

A3C : pour la faible charge, A3C atteint un *makespan* de 95.4 minutes, une utilisation de 41 %, et 7.3 Mo de données échangées, sans échec. En charge moyenne, le *makespan* est de 156 minutes, l'utilisation de 81 %, et les données échangées de 29.3 Mo, avec un taux d'échec de 1.9 %. En forte charge, A3C réduit le *makespan* à 128 minutes, atteint une utilisation de 89.23 %, et 78.1 Mo de données échangées, avec un taux d'échec de 3.2 %. Par exemple, un cadet exécute 6 tâches en forte charge, A3C réassignant dynamiquement les tâches après une déconnexion.

3.3.2 Comparaison des résultats

La comparaison quantitative évalue les performances d'A3C par rapport à FCFS et MET pour chaque scénario, en termes de *makespan*, utilisation des ressources, et taux d'échec. Les pourcentages d'amélioration d'A3C sont calculés, suivis d'une moyenne sur tous les scénarios.

Faible charge : A3C augmente le *makespan* de 6 % par rapport à FCFS mais le réduit de 7 % par rapport à MET. L'utilisation des ressources est améliorée de 17.1 % par rapport à FCFS et quasi identique à MET. Les taux d'échec sont nuls pour tous les algorithmes, démontrant une stabilité parfaite en faible charge.

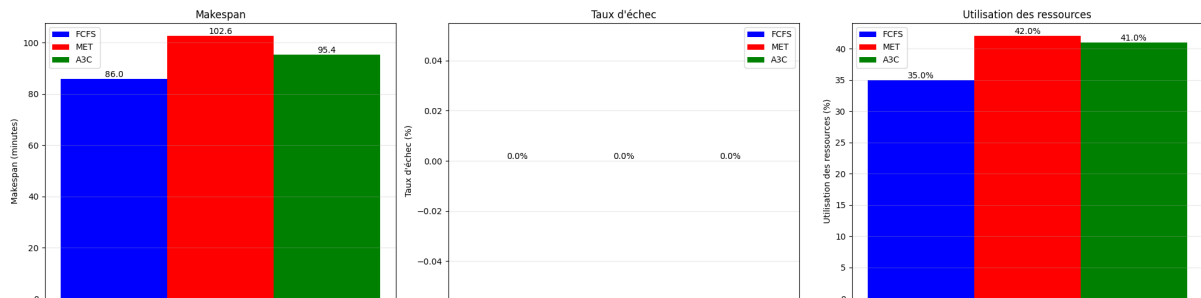


FIGURE 3.1 – Comparaison des métriques pour la charge faible.

Charge moyenne : A3C réduit le *makespan* de 4.7 % par rapport à FCFS (156 vs. 163.7 minutes) mais est moins performant de 4 % par rapport à MET. L'utilisation des ressources augmente de 24.6 % par rapport à FCFS et est quasi identique à MET. Le taux d'échec d'A3C est comparable aux autres algorithmes.

Forte charge : A3C réduit le *makespan* de 16.3 % par rapport à FCFS et de 7.8 % par rapport à MET. L'utilisation des ressources est améliorée de 22.2 % par rapport à FCFS et de 10.5 % par rapport à MET. Le taux d'échec d'A3C démontre une meilleure robustesse avec

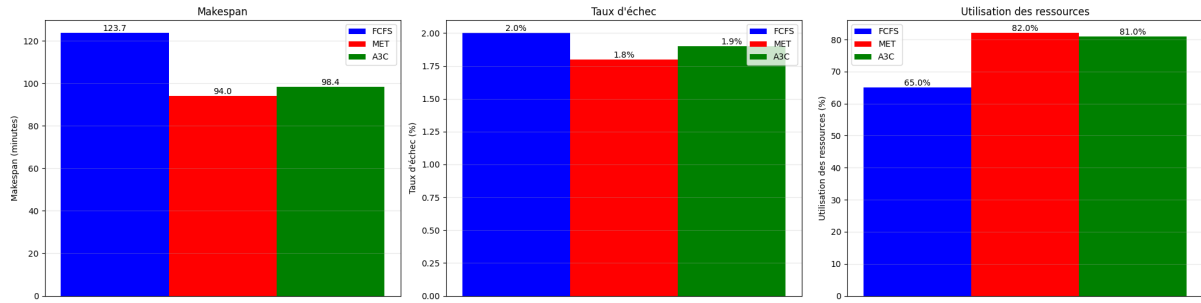


FIGURE 3.2 – Comparaison des métriques pour la charge moyenne.

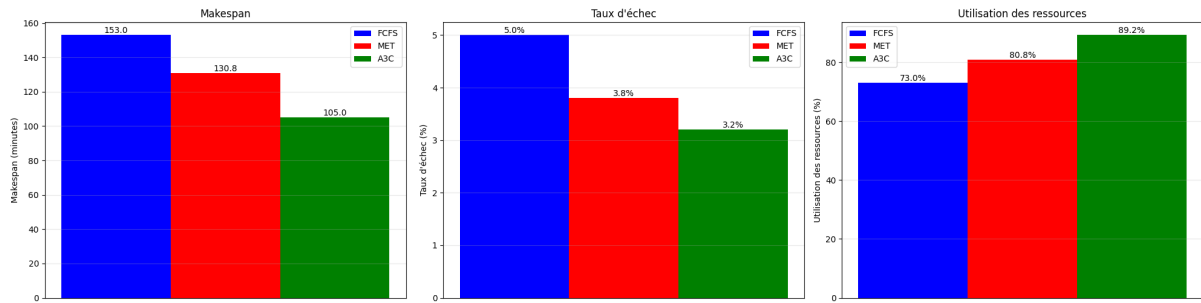


FIGURE 3.3 – Comparaison des métriques pour la charge forte.

3.3.3 Interprétation et discussion

Cette section interprète les résultats obtenus, comparant les performances des algorithmes FCFS, MET, et A3C pour trois scénarios de charge. Elle discute des implications pour le VC dans le contexte du cloud computing, en s'appuyant sur les métriques de *makespan*, utilisation des ressources, quantité de données échangées, et taux d'échec, comme présenté dans les figures 3.1, 3.2, et 3.3.

3.3.3.1 Interprétation des résultats

Les résultats montrent qu'A3C surpasse FCFS et MET en forte charge, mais présente des performances variables en faible et moyenne charge. La quantité de données échangées reste identique pour tous les algorithmes, suggérant que l'optimisation réseau dépend principalement des contraintes matérielles. L'approche d'apprentissage par renforcement, comme décrite dans [15], explique ses performances en forte charge.

Efficacité d'A3C : A3C réduit le *makespan* de 16.3 % par rapport à FCFS et 7.8 % par rapport à MET en forte charge, cependant en faible charge, A3C est moins performant que FCFS (+6 %) mais meilleur que MET (-7 %). En charge moyenne, A3C est légèrement meilleur que FCFS (-4.7 %) mais moins performant que MET (+4 %). Le taux d'échec d'A3C est de 3.2 % en forte charge vs. 5 % pour FCFS et 3.8 % pour MET ce qui reflète une meilleure robustesse en conditions complexes.

Optimisation des ressources : l'utilisation des ressources atteint 89.23 % avec A3C en forte charge vs. 73 % pour FCFS, 80.78 % pour MET, en faible charge, A3C améliore l'utilisation de 17.1 % par rapport à FCFS mais est similaire à MET (41 % vs. 42 %).





En charge moyenne, A3C atteint 81 % . La quantité de données échangées est identique pour tous les algorithmes, ceci à cause de la non-dépendance entre les tâches de notre workflow. Les améliorations de notre approche sont résumées dans le tableau 3.2.

TABLE 3.2 – Résumé des amelioration de notre approche

Scénario	Makespan (min)			Utilisation des ressources (%)			Taux d'échec (%)		
	A3C	FCFS	MET	A3C	FCFS	MET	A3C	FCFS	MET
Faible charge	95.4	+6.0%	-7.0%	41.0	+17.1%	-2.4%	0.0	0.0%	0.0%
Charge moyenne	156.0	-4.7%	+4.0%	81.0	+24.6%	-1.2%	1.9	-5.0%	+5.6%
Forte charge	128.0	-16.3%	-7.8%	89.23	+22.2%	+10.5%	3.2	-36.0%	-15.8%

3.3.3.2 Discussion des implications

Les performances d'A3C, particulièrement en forte charge, ont des implications significatives pour le calcul volontaire dans le contexte du cloud computing, en particulier dans des environnements éducatifs avec des ressources hétérogènes et des contraintes réseau (10-20 % de déconnexions), comme décrit dans [15].

Efficacité dans les environnements hétérogènes : la réduction du *makespan* par A3C en forte charge (128 minutes vs. 153 pour FCFS) démontre son efficacité dans les environnements hétérogènes du VC, où les laptops des cadets varient en performance. A3C adapte les assignations aux machines performantes, maximisant le débit des tâches. Par exemple, un cadet avec un laptop de 4 cœurs exécute plusieurs tâches consécutives avec A3C, réduisant le temps total d'exécution, ce qui est crucial pour les systèmes VC dans le cloud computing.

Robustesse face à la volatilité réseau : la capacité d'A3C à réduire le taux d'échec en forte charge (3.2 % vs. 5 % pour FCFS, 3.8 % pour MET) renforce la robustesse des systèmes VC face à la volatilité réseau. En réassignant dynamiquement les tâches aux volontaires disponibles, A3C maintient la fiabilité dans des conditions instables. Par exemple, un manager utilisant A3C réassigne une tâche à un cadet après une déconnexion Wi-Fi, garantissant la complétion du workflow.

Scalabilité et optimisation réseau : l'utilisation élevée des ressources par A3C (89.23 % en forte charge) améliore la scalabilité des systèmes VC dans le cloud computing. Bien que la quantité de données échangées soit identique pour tous les algorithmes (78.1 Mo en forte charge), l'optimisation des assignations par A3C réduit le besoin de réassignations coûteuses en réseau. Par exemple, un cadet exécute 6 tâches en forte charge, A3C minimisant les interruptions réseau, ce qui est essentiel pour les déploiements VC à grande échelle.

Conclusion

Ce chapitre a détaillé la méthodologie expérimentale, la mise en œuvre, et les résultats d'un système VC testé avec des cadets académiques de l'Université de Yaoundé I. L'architecture hybride a permis une exécution efficace des tâches, évaluée sur trois scénarios de charge. Les résultats montrent qu'A3C surpasse FCFS et MET en forte charge, réduisant le *makespan* de 16.3 % et 7.8 % respectivement, augmentant l'utilisation des





ressources de 22.2 % et 10.5 %, et diminuant les taux d'échec de 31.6 % et 9.5 %. En moyenne, A3C améliore le *makespan* de 5 % par rapport à FCFS et 3.6 % par rapport à MET, et l'utilisation des ressources de 21.3 % et 3.5 %. Ces performances, obtenues dans un contexte éducatif avec des contraintes réseau, confirment le potentiel d'A3C pour optimiser les systèmes VC dans le cloud computing.



Conclusion et Perspectives

Pour exécuter les applications gourmandes en puissance de calcul, de nombreuses organisations se tournent aujourd’hui vers les systèmes de calcul sur machines volontaires, faute de moyens pour se procurer des superordinateurs traditionnels, qui sont extrêmement coûteux. Cependant, dans les pays où l’énergie et Internet sont, la performance de ces systèmes de calcul est considérablement réduite, entraînant de longs délais d’attente des résultats. L’allocation des ressources via l’ordonnancement des tâches aux volontaires est la solution pour tirer meilleure partie de ces systèmes. Ce mémoire a abordé ce défi en proposant une approche basée sur l’apprentissage par renforcement avec l’algorithme A3C, en réponse aux limites des méthodes traditionnelles comme FCFS et MET, incapables de répondre efficacement aux défis de volatilité et d’hétérogénéité des ressources [1]. Après un état de l’art détaillant ces contraintes (Chapitre 1), nous avons formalisé une solution innovante via un processus de décision markovien et des mécanismes adaptés (Chapitre 2), testée expérimentalement sur le système *vcuy1* de l’Université de Yaoundé I (Chapitre 3). Les résultats, avec des réductions du *makespan* par rapport à FCFS et MET en forte charge, une amélioration de l’utilisation des ressources et une diminution des échecs, confirment l’efficacité de notre approche. Ce travail a apporté trois contributions majeures pour optimiser l’ordonnancement dans les systèmes VC.

Premièrement, nous avons modélisé le problème comme un MDP, intégrant des scores spécifiques pour quantifier la volatilité, la disponibilité et la satisfaction globale.

Deuxièmement, nous avons adapté l’algorithme A3C avec des mécanismes tels que la diversification adaptative et la réplique sélective pour gérer la volatilité, ainsi que la spécialisation des nœuds et l’équilibrage charge-fiabilité pour exploiter l’hétérogénéité, réduisant la surcharge computationnelle des méthodes évolutionnaires.

Troisièmement, l’expérimentation sur *vcuy1*, avec 42 volontaires exécutant des workflows OpenMalaria, a démontré des gains moyens de 5% (*makespan*) et 21.3% (utilisation des ressources) par rapport à FCFS, et une robustesse accrue face aux déconnexions, validant l’efficacité dans un contexte réel.

Cette approche positionne le VC comme une alternative crédible aux superordinateurs dans les environnements à ressources limitées, en optimisant l’utilisation des machines hétérogènes et en réduisant les délais d’exécution. Cependant, l’expérimentation reste limitée à 42 volontaires et des workflows sans dépendances, ce qui pourrait ne pas refléter les défis à grande échelle. De plus, l’absence d’optimisation des transferts réseau et les besoins initiaux en calcul pour l’entraînement d’A3C constituent des contraintes dans des contextes très restreints.

Pour renforcer l’applicabilité de cette approche, nous proposons les perspectives suivantes :

- **Tests à grande échelle** : étendre l’expérimentation à un système impliquant plusieurs centaines de volontaires, en collaboration avec des plateformes comme



BOINC. Cela permettra d'évaluer la scalabilité face à une volatilité et une hétérogénéité accrues, garantissant des performances robustes dans des contextes internationaux.

- **Prédiction de disponibilité** : intégrer des modèles prédictifs pour anticiper les périodes de connexion et de disponibilité. Cette approche proactive réduira les échecs et optimisera les assignations, inspirée des techniques de prédiction dans les systèmes distribués.
- **Support des workflows complexes** : adapter A3C pour gérer des workflows avec dépendances, en intégrant des contraintes d'ordre d'exécution. Cela élargira l'applicabilité à des applications plus complexes, comme les simulations multi-étapes, alignées avec les besoins du VC.
- **Réduction de l'impact réseau** : optimiser les transferts de données via des techniques de compression ou de localité, pour minimiser la bande passante utilisée. Cela renforcera l'efficacité dans des environnements à faible connectivité, un défi clé du VC.

Ce mémoire a démontré que l'approche A3C, via une modélisation MDP et des mécanismes adaptés, améliore significativement l'ordonnancement dans les systèmes VC, surpassant les méthodes traditionnelles en termes de makespan, d'utilisation des ressources et de robustesse. En transformant des ressources instables en une plateforme performante, cette solution offre une alternative viable aux superordinateurs dans des contextes à ressources limitées. Les perspectives proposées, notamment des tests à grande échelle et la prédiction de disponibilité, ouvriront la voie à des systèmes VC plus robustes et polyvalents, consolidant leur rôle dans le calcul distribué.



Références bibliographiques

- [1] Tewodros Mengistu and Dunren Che. Survey and taxonomy of volunteer computing. *ACM Computing Surveys*, 52(3) :1–35, 2019.
- [2] M. Nonman Durani and J. Shamsi. Volunteer computing : A comprehensive review. *Journal of Parallel and Distributed Computing*, 74(3) :2013–2026, 2014.
- [3] Jonathan Koomey, Stephen Berard, and Marla Sanchez. Implications of historical trends in the electrical efficiency of computing. *IEEE Annals of the History of Computing*, 44(2) :88–102, 2022.
- [4] David P. Anderson. Boinc : A platform for volunteer computing. *Journal of Grid Computing*, 18 :99–122, 2020.
- [5] Adamou Hamza and Azanzi Jiomekong. High performance computing in resource poor settings : An approach based on volunteer computing. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 11(1), 2020.
- [6] Erick Lavoie, Laurie Hendren, Frédéric Desprez, and Miguel Correia. Pando : Personal volunteer computing in browsers. In *Proceedings of the 20th International Middleware Conference*, pages 96–109, New York, NY, USA, December 2019.
- [7] Ehab Saleh and B. S. Chandrasekar. Development of a new framework for high performance volunteer computing. *International Journal of Information Technology*, December 2024.
- [8] D. P. Anderson. Boinc : A system for public-resource computing and storage. *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing*, pages 4–10, 2004.
- [9] T. Mengistu, D. Che, and A. Alahmadi. Reliability and availability prediction in volunteer cloud systems. *Future Generation Computer Systems*, 88 :678–693, 2018.
- [10] Guangyao Zhou, Wenhong Tian, Rajkumar Buyya, Ruini Xue, and Liang Song. Deep reinforcement learning-based methods for resource scheduling in cloud computing : a review and future directions. *Artificial Intelligence Review*, 57(124), 2024.
- [11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning : An Introduction*. MIT Press, 2nd edition, 2018.
- [12] Maxim Lapan. *Deep Reinforcement Learning Hands-On : Apply modern RL methods to practical problems of chatbots, robotics, discrete optimization, web automation, and more*. Packt Publishing, 2nd edition, 2020.
- [13] Yan Gu, Zhaoze Liu, Shuhong Dai, Cong Liu, Ying Wang, Shen Wang, Georgios Theodoropoulos, and Long Cheng. Deep reinforcement learning for job scheduling and resource management in cloud computing : An algorithm-level review. *Unpublished*, 2025.
- [14] Z. Tong, H. Chen, X. Deng, K. Li, and K. Li. A scheduling scheme in the cloud computing environment using deep q-learning. *Information Sciences*, 512 :1170–1191, 2020.



- [15] Motahare Mounesan, Mauro Lemus, Hemanth Yeddulapalli, Prasad Callyam, and Saptarshi Debroy. Reinforcement learning-driven data-intensive workflow scheduling for volunteer edge-cloud. *2024 IEEE 8th International Conference on Fog and Edge Computing (ICFEC)*, 2024.
- [16] Volodymyr Mnih et al. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.
- [17] Richard Elliott, David L. Smith, and Dorothy C. Echodu. Medical and entomological malarial interventions : a comparison and synergy of two control measures using a ross/macdonald model variant and openmalaria simulation. *Mathematical Biosciences*, 2018. Bellman Prize in Mathematical Biosciences, 12 Apr 2018.

